

华北理工大學

NORTH CHINA UNIVERSITY OF SCIENCE AND TECHNOLOGY

《 数 据 管 理 系 统 》

课 程 论 文

题目：智慧图书馆数字资源平台

项目成员：王广驰 **202414870316**

郭瀚铭 **202414870108**

刘金鹤 **202414870317**

专业班级：24 级数据科学与大数据技术 3 班

24 级数据科学与大数据技术 1 班

学　　院：理学院

指导教师：宁学斌

2026 年 11 月 10 日

目录

1 项目概述.....4
1.1 项目背景.....4
1.2 项目建设目标.....4
1.3 项目主要内容与技术路线.....4
1.4 项目实施环境与成果.....4
1.5 项目分工与进度.....5
2. 技术选型论证.....5
2.1 关系型数据库与 NoSQL 数据库对比.....5
2.2 NoSQL 数据库类型比较与选择.....6
2.3 基于 CAP 理论的业务场景分析.....6
2.4 BASE 理论及最终一致性分析.....7
2.5 技术选型结论及潜在风险.....7
3 数据模型设计.....7
3.1 数据库逻辑结构设计.....7
3.2 字段类型设计.....8
3.3 集合设计与关键决策.....9
3.4 索引设计.....9
4 功能实现.....10
4.1 数据导入与检索功能.....10
4.2 聚合统计功能.....10
4.3 索引优化与性能分析.....11
4.4 用户权限管理.....12
4.5 Python 应用程序实现.....12
5 文件存储：GridFS 存储结构与上传下载实现.....12
5.1 存储结构.....12
5.2 上传实现.....13
5.3 下载与校验.....13
6 架构演进：副本集部署与故障转移、分片集群与分片键论证.....14
6.1 演进路线.....14
6.2 副本集部署与故障转移.....14
6.3 分片集群部署.....15

6.4 数据分布与查询路由 15
6.5 分片键论证 16
6.5 分片键论证 16
7 问题与总结 17
7.1 技术难点排查 17
7.2 项目反思 18
附录　过程记录截图 19
附录 A　T3 单机实现 19
附录 B　T4 应用开发 26
附录 C　T5 副本集 32
附录 D　T6 分片集群 36
附录 E　T7 验收 40

1 项目概述

1.1 项目背景

本项目以某高校图书馆数字资源管理为业务背景。随着纸质馆藏规模扩大和电子资源增加，平台需要统一管理图书、读者、借阅流水及 PDF 电子书、PNG 封面等数据。根据任务书设定，图书馆纸质馆藏约 50 万册，上述数据在字段结构、嵌套关系和存储形式上存在差异，借阅记录也具有持续增长的特点。为满足异构数据组织、信息检索、统计分析和数字资源管理需求，本项目采用 MongoDB 作为核心数据库，结合 Python 开发智慧图书馆数字资源平台。

1.2 项目建设目标

本项目旨在构建集馆藏管理、借阅统计和数字资源存储于一体的数据库应用平台。主要目标包括：完成 MongoDB 技术选型及图书、读者、借阅流水等数据模型设计；实现信息管理、多条件检索、聚合统计、索引优化和权限控制；基于 Python 和 PyMongo 开发应用程序，利用 GridFS 实现数字资源上传、下载与完整性验证；部署副本集和分片集群，验证数据复制、故障切换及查询路由机制。

1.3 项目主要内容与技术路线

项目按照需求分析与技术选型、数据模型设计、单机功能实现、Python 应用与 GridFS 开发、分布式架构验证五个阶段开展，形成从数据建模、业务功能实现到分布式集群测试的完整技术路线。

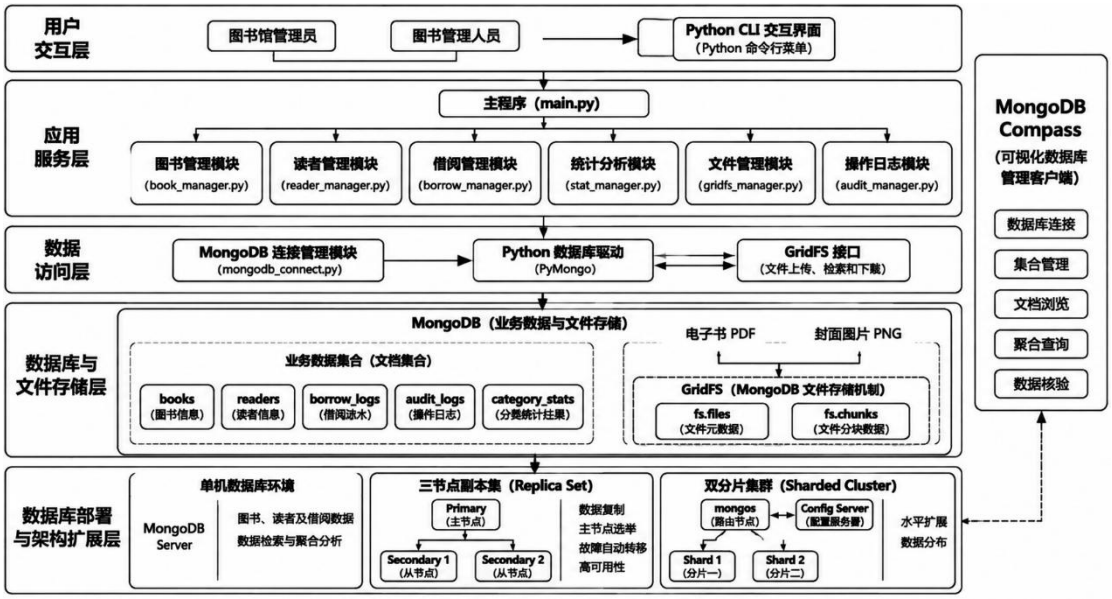


图 1-1 智慧图书馆数字资源平台总体技术架构图

1.4 项目实施环境与成果

项目采用 CentOS 虚拟机作为数据库服务器，使用 MongoDB 6.0.29，通过

MongoDB Compass 进行可视化管理，以 Python 和 PyMongo 实现应用开发。测试数据包括 50 条图书记录、30 条读者记录、3000 条借阅流水及 6 个数字资源文件。最终完成图书与读者管理、借阅检索、统计分析、文件管理和操作日志等功能，并完成 GridFS 文件验证、三节点副本集及双分片集群实验。

1.5 项目分工与进度

项目分工

本项目围绕智慧图书馆数字资源平台的建设需求，按照需求分析、数据库设计、功能开发、分布式部署和系统验收等环节开展工作。

成员	分工
王广驰	架构与运维
郭瀚铭	数据与程序
刘金鹤	文档与验收

项目进度

阶段	主要工作
第一阶段	项目需求分析与 MongoDB 技术选型
第二阶段	数据库逻辑结构设计、字段类型及索引规划
第三阶段	数据导入、检索统计及权限管理功能实现
第四阶段	Python 应用程序开发与 GridFS 文件存储
第五阶段	三节点副本集及双分片集群部署与测试
第六阶段	系统功能验收、实验结果整理及报告撰写

2．技术选型论证

2.1 关系型数据库与 NoSQL 数据库对比

传统关系型数据库通常采用预定义表结构组织数据，通过主键、外键及关联查询维护业务实体之间的关系，具有事务处理成熟、结构规范等优势。本项目中的图书和读者信息存在部分可选字段，馆藏位置具有嵌套结构，若采用关系模型，需要根据业务需求设计相应的扩展字段及关联关系。

关系型数据库与 NoSQL 数据库在数据模型、扩展方式和事务机制等方面存在差异，具体如表 2-1 所示。

表 2-1 关系型数据库与 NoSQL 数据库对比

比较维度	关系型数据库	NoSQL 数据库
数据模型	以二维表为核心，通常具有预定义结构	包含键值、文档、列族、图等多种模型
数据结构	字段结构相对固定	可根据业务采用灵活的数据结构
关联方式	主要通过主外键及 JOIN 实现	根据数据库类型采用嵌套、引用或图关联等方式

扩展方式	通常以纵向扩展为主，也支持分布式方案	多种类型支持水平扩展
事务机制	通常提供成熟的 ACID 事务支持	不同产品机制不同，部分支持多文档事务
一致性	通常强调事务一致性和数据完整性	可根据产品及配置采用不同的一致性策略
适用场景	结构稳定、关联复杂、事务要求较高的业务	数据结构多样、数据增长较快及分布式应用场景

本项目采用嵌套文档组织馆藏位置，利用数组保存图书标签及读者偏好，借阅流水则通过 ISBN 和读者证号与相关业务数据建立逻辑关联，从而使数据结构与业务对象保持较直接的对应关系。

2.2 NoSQL 数据库类型比较与选择

NoSQL 数据库并非单一的数据存储技术，根据数据组织形式和应用特点，主要包括键值数据库、文档数据库、列族数据库和图数据库，其特点及适用场景如表 2-2 所示。

表 2-2 四类 NoSQL 数据库技术对比

数据库类型	主要特点	典型产品	典型应用场景
键值数据库	以键值对形式存储数据，结构简单，读写效率较高	Redis、Riak	缓存、会话管理、快速状态查询
文档数据库	以文档形式存储数据，支持嵌套结构和数组，数据模型灵活	MongoDB、CouchDB	内容管理、商品信息、数字资源管理
列族数据库	按列族组织数据，适合大规模分布式数据存储和高吞吐访问	HBase、Cassandra	海量日志、时序数据、分布式数据分析
图数据库	以节点、边和属性描述实体及其关系，擅长复杂关联查询	Neo4j、JanusGraph	社交网络、知识图谱、关系推荐

综合比较可知，文档数据库在嵌套结构表达、数组管理及多字段查询方面更符合本项目需求。因此，选择 MongoDB 作为核心数据库管理系统。

2.3 基于 CAP 理论的业务场景分析

CAP 理论指出，在分布式系统发生网络分区时，一致性（Consistency）与可用性（Availability）无法同时得到无条件保证，分区容错性（Partition Tolerance）则是系统应对网络分区的能力。

（1）馆藏查询：优先考虑可用性与分区容错性（AP 倾向）

对于普通馆藏查询，短暂的数据副本延迟通常不会影响基本的信息浏览，因此可以采用 secondaryPreferred 读取偏好，在适当情况下优先从从节点读取数据，提高查询服务的可用性。但涉及实时库存和借阅状态时，应优先读取主节点。**（2）借阅流水写入：优先考虑一致性与分区容错性（CP 倾向）**

借阅流水涉及图书借出、归还及逾期等关键业务，应优先保证数据可靠性。MongoDB 可采用主节点写入及 majority 写关注级别，使写入获得多数节点确认；当无法满足多数派确认条件时，相关写入可能失败或等待，以避免错误确认关键业务数据。两类业务的 CAP 取舍如表 2-3 所示。

表 2-3 图书馆业务场景下的 CAP 取舍

业务场景	优先考虑的特性	主要原因	实现策略
普通馆藏信息查询	A、P（AP 倾向）	查询连续性较重要，可容忍短暂副本延迟	合理配置 secondaryPreferred 等读取偏好
借阅流水写入	C、P（CP 倾向）	必须保证借阅记录准确，避免冲突写入	Primary 写入，采用 majority 写关注
借阅状态与库存核对	C 优先	需要保证业务状态准确	优先读取主节点，控制写入确认
统计分析	可接受适度延迟	统计结果不要求操作后立即刷新	基于已提交数据进行周期性统计

2.4 BASE 理论及最终一致性分析

BASE 理论包括基本可用（Basically Available）、软状态（Soft State）和最终一致性（Eventually Consistent），强调系统可以在一定程度上接受数据状态的暂时差异，并通过后续同步实现最终一致。

在本项目中，图书分类统计、学院借阅排行和借阅趋势主要用于辅助管理决策，可以通过 MongoDB 聚合管道定期更新，允许统计结果与最新借阅流水之间存在短暂时间差。但借阅流水写入、库存扣减及归还状态更新等核心业务，应优先保证数据准确性，不宜简单采用最终一致性策略。

2.5 技术选型结论及潜在风险

综合业务需求及不同数据库技术的适用特点，本项目选择 MongoDB 作为核心数据库管理系统，以满足异构数据管理、业务查询、统计分析及分布式部署需求。

本平台选择 MongoDB，能够统一管理图书、读者、借阅记录和数字资源，支持灵活的信息检索与业务统计，并通过多节点部署提高服务连续性。针对借阅和库存等关键业务，还将配合数据校验和权限管理，保障业务信息的准确性。

3 数据模型设计

3.1 数据库逻辑结构设计

本项目创建 smartlib 数据库，采用文档模型组织业务数据，主要包含图书、读者、借阅流水和操作日志四个基础集合，并设置统计结果集合及 GridFS 文件存储区。其逻辑结构如图 3-1 所示：

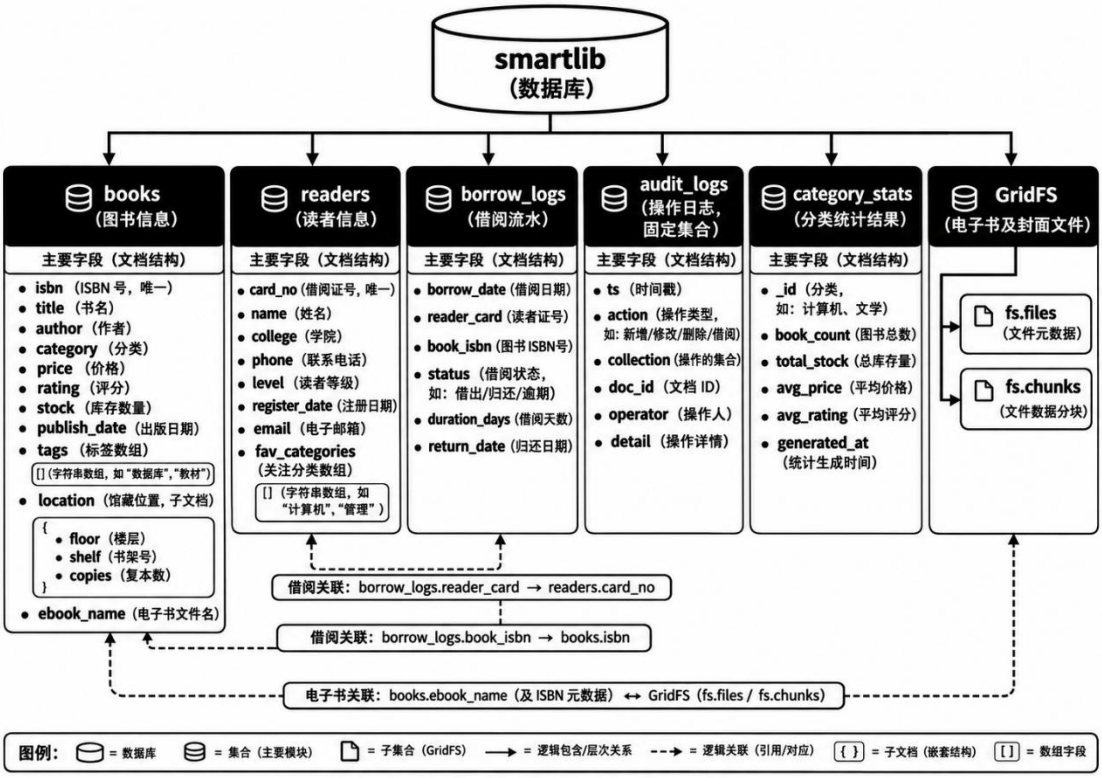


图 3-1 智慧图书馆数据库逻辑结构图

其中，前三个集合存储核心业务数据，audit_logs 记录关键操作，category_stats 保存聚合统计结果，GridFS 用于管理 PDF 电子书及 PNG 封面图片。

3.2 字段类型设计

根据业务需求确定各集合的 BSON 数据类型，具体如表 3-1 所示。

表 3-1 核心集合字段类型设计

集合	字段及 BSON 类型
books	isbn、title、author、publisher、category、ebook_name：String；price、rating：Double；stock、borrow_count：Int；publish_date：Date；tags：Array；location：Object
readers	card_no、name、college、phone、level：String；email：String/Null；register_date：Date；fav_categories：Array
borrow_logs	reader_card、book_isbn、status：String；borrow_date、return_date：Date；duration_days：Int
audit_logs	ts：Date；action、collection、doc_id、operator：String；detail：Object
category_stats	_id：String；book_count、total_stock：Int；avg_price、avg_rating：Double；generated_at：Date

其中，location 采用嵌套文档，包含 floor、shelf 和 copies 三个字段；tags、fav_categories 及 return_date 根据实际业务情况允许缺省，email 允许为空值。

3.3 集合设计与关键决策

本项目主要采用以下设计方案：

（1）**字段类型选择：** ISBN 和借阅证号采用 String，避免标识符参与数值运算；价格和评分采用 Double，支持数值比较与统计；日期统一采用 Date，便于时间范围检索和聚合分析。

（2）**嵌套文档设计：** 将楼层、架位和复本数嵌入 books.location，便于一次查询获取完整馆藏位置。若位置需要独立管理或被多个业务实体共享，则可拆分为独立集合。

（3）**借阅流水设计：** borrow_logs 采用普通集合，而非时间序列集合。由于归还和逾期处理需要修改记录状态及归还时间，普通集合更符合实际业务操作，也便于后续分片部署。

（4）**操作日志设计：** audit_logs 采用固定集合（Capped Collection），设置容量 16 MB、最大文档数 100000 条，利用自动淘汰机制控制日志规模，并满足最近 100 条操作记录的查询需求。

（5）**数据关联与文件存储：** 借阅流水通过 reader_card 和 book_isbn 关联读者与图书，减少重复数据；电子书和封面图片采用 GridFS 存储，通过 ISBN 及文件元数据与图书信息建立关联。

3.4 索引设计

根据实际业务查询需求，为各集合规划索引，具体如表 3-2 所示。

表 3-2 数据库索引设计

集合	索引字段	索引类型及用途
books	isbn	唯一索引，保证 ISBN 唯一
books	category、price	复合索引，支持组合筛选
books	tags	多键索引，支持标签查询
books	rating	降序索引，支持评分排序
books	publish_date	单字段索引，支持日期范围查询
readers	card_no	唯一索引，保证借阅证号唯一
readers	college	单字段索引，支持学院查询
borrow_logs	reader_card、borrow_date	复合索引，支持读者借阅记录查询
borrow_logs	book_isbn、borrow_date	复合索引，支持图书借阅记录查询
borrow_logs	borrow_date	单字段索引，支持时间范围检索
audit_logs	ts	降序索引，支持最近操作日志查询

此外，MongoDB 自动为各集合的_id 字段建立唯一索引。上述索引与业务查

询需求相对应，为后续检索效率优化及分布式部署提供基础。

4 功能实现

4.1 数据导入与检索功能

完成数据库结构及索引设计后，首先将配套 JSON 数据导入 MongoDB，并将日期字符串转换为 BSON Date 类型。导入后核对数据数量，最终得到 50 条图书记录、30 条读者记录和 3000 条借阅流水，为后续功能测试提供数据基础。

在此基础上，利用 PyMongo 实现六项图书检索功能，具体如表 4-1 所示。

表 4-1 图书检索功能实现

编号	检索功能	实现方式	实际结果数
R1	类别与价格区间组合筛选	条件查询、$gte、$lte	8
R2	多标签同时匹配	$all	1
R3	书名模糊检索	$regex、忽略大小写	2
R4	出版年份范围检索	Date 日期范围查询	23
R5	查询缺少 tags 字段的图书	$exists	4
R6	评分 Top10	sort()、limit()	10

其中，类别与价格检索通过组合查询条件实现多字段筛选；标签检索要求图书同时包含指定标签；书名检索采用正则表达式实现模糊匹配；年份检索通过日期上下界限定查询范围；缺失字段检索用于识别数据不完整的图书；评分排行则按照评分降序排列，并以累计借阅次数作为辅助排序条件。

通过上述功能，实现了图书信息的多条件检索及数据质量核查。

4.2 聚合统计功能

针对图书馆日常管理与借阅分析需求，利用 MongoDB Aggregation Pipeline 实现五项统计功能，具体如表 4-2 所示。

表 4-2 聚合统计功能实现

编号	统计功能	主要操作
S1	图书分类统计	$group、$avg、$sum
S2	学院借阅量 Top5	$lookup、$unwind、$group
S3	年月借阅趋势	$dateToString、$group
S4	借阅量 Top10 及馆藏位置	$group、$sort、$lookup
S5	分类统计结果保存	$merge

分类统计按照图书类别分组，计算各类别藏书量、库存总量、平均价格及平均评分；学院借阅排行通过$lookup 关联读者集合，获取读者所属学院后进行分组统计；年月趋势利用$dateToString 提取借阅日期中的年月信息，形成 2023 年 1 月至 2025 年 12 月共 36 个月的统计结果。

对于借阅量排行，首先按照图书 ISBN 统计借阅次数，再关联图书集合获取书名和馆藏位置。最后，利用$merge 将分类统计结果写入 category_stats 集合，生成 6 条分类统计记录，便于后续查询和使用。

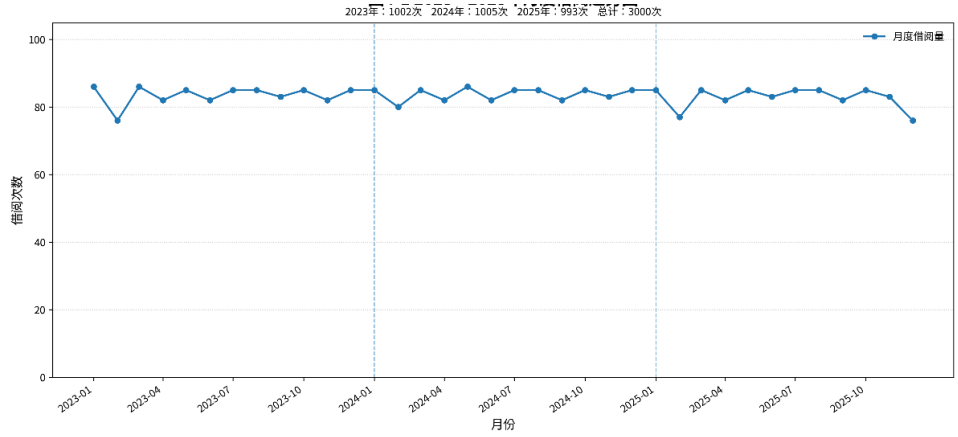


图 4-1 2023—2025 年月度借阅趋势图

表 4-3 图书分类聚合统计结果

图书类别	图书数量	总库存	平均价格	平均评分
计算机	15	172	89.53	4.53
文学	12	181	47.02	4.62
经济管理	7	64	90.86	4.30
数学	6	122	40.67	4.43
历史	5	50	95.60	4.66
外语	5	72	59.00	4.32
合计	50	661	—	—

4.3 索引优化与性能分析

为验证索引设计的实际效果，选取类别与价格组合查询、读者及借阅时间查询两种典型场景，分别在建立索引前后执行查询，并通过 explain("executionStats") 分析执行计划。

表 4-4 索引优化前后执行计划对比

查询场景	有索引	无索引
类别与价格查询	IXSCAN，检查 8 个索引键	COLLSCAN，扫描 50 条文档
读者与借阅时间查询	IXSCAN，检查 66 个索引键	COLLSCAN，扫描 3000 条文档

实验结果表明，建立索引后，两项查询均采用索引扫描（IXSCAN），而未建立索引时采用集合扫描（COLLSCAN），减少了不必要的文档扫描。其中，借阅流水查询的数据规模为 3000 条，索引能够有效缩小查询范围。

此外，为进一步观察数据规模对查询性能的影响，使用 20 万条构造数据开展补充测试。实验中，无索引查询耗时约 51 ms，有索引查询耗时约 2 ms，表明在数据规模增大时，合理的索引设计能够降低查询开销。需要说明的是，原始 50 条图书和 3000 条借阅流水规模较小，执行计划中的耗时均显示为 0 ms，因此主

要通过扫描文档数量及执行阶段判断索引效果。

4.4 用户权限管理

为保障数据库访问安全，根据不同业务角色设置只读、读写和管理员三类账号，具体如表 4-4 所示。

表 4-5 数据库用户权限设计

用户账号	权限角色	权限范围
app_ro	read	仅允许读取 smartlib 数据库
app_rw	readWrite	允许读取和修改 smartlib 数据库
root	root	具有数据库管理权限

通过权限验证实验，使用只读账号查询图书集合，能够正常获取 50 条图书记录；进一步尝试执行新增图书操作时，MongoDB 返回 not authorized 错误，说明权限控制能够有效阻止未授权写入。读写账号和管理员账号则分别承担日常业务操作及数据库管理任务。

4.5 Python 应用程序实现

为提高系统的可操作性，采用 Python 与 PyMongo 开发菜单式应用程序，并按照功能职责进行模块化设计。程序由九个 Python 源文件组成，主要包括数据库连接、数据初始化、图书管理、读者管理、借阅管理、统计分析、GridFS 文件管理、审计日志及主程序模块。

其中，mongodb_connect.py 负责统一数据库连接，setup_smartlib.py 完成集合创建、索引建立、数据导入及初始化核验；book_manager.py、reader_manager.py 和 borrow_manager.py 分别实现图书检索、读者管理及借阅归还操作；stat_manager.py 负责聚合统计，audit_manager.py 记录关键业务操作，主程序通过菜单调用各功能模块。

程序主菜单共设置 16 项功能，涵盖图书检索、统计分析、电子资源管理、审计日志、借阅登记、图书归还、读者增删改查及分页浏览等操作。通过统一连接模块和模块化调用，实现了数据库操作与业务功能的分离，便于程序维护和后续扩展。

综上，本项目完成了 MongoDB 数据检索、聚合统计、索引优化、用户权限控制及 Python 应用程序开发，实现了从数据库基础操作到业务应用的功能整合。

5 文件存储：GridFS 存储结构与上传下载实现

5.1 存储结构

平台需要保存 3 本电子书（PDF）和 3 张封面图（PNG）。这些文件若直接以二进制字段写入业务文档，会受 BSON 单文档 16MB 上限约束，也不便于流式读写，因此统一交给 GridFS 管理。

GridFS 以“桶”为单位组织文件。本平台使用默认桶名 fs，对应两个集合：fs.files 保存文件元数据，主要字段有_id、filename（原始文件名）、length（字节

数）、uploadDate、chunkSize（默认 255KB）与自定义 metadata；fs.chunks 保存文件内容，每个 chunk 一条文档，用 files_id 关联文件、用 n 递增编号、用 data 存放二进制数据。

自定义 metadata 是文件与业务数据关联的关键：isbn 指向 books 集合中的图书，kind 取 ebook 或 cover，original_name 保留上传时的文件名，bytes 与 md5 记录上传时的字节数和摘要。books 文档中的 ebook_name 保存电子书文件名，再通过 isbn 与 metadata.isbn 对应，图书记录与文件因此可以互相定位。

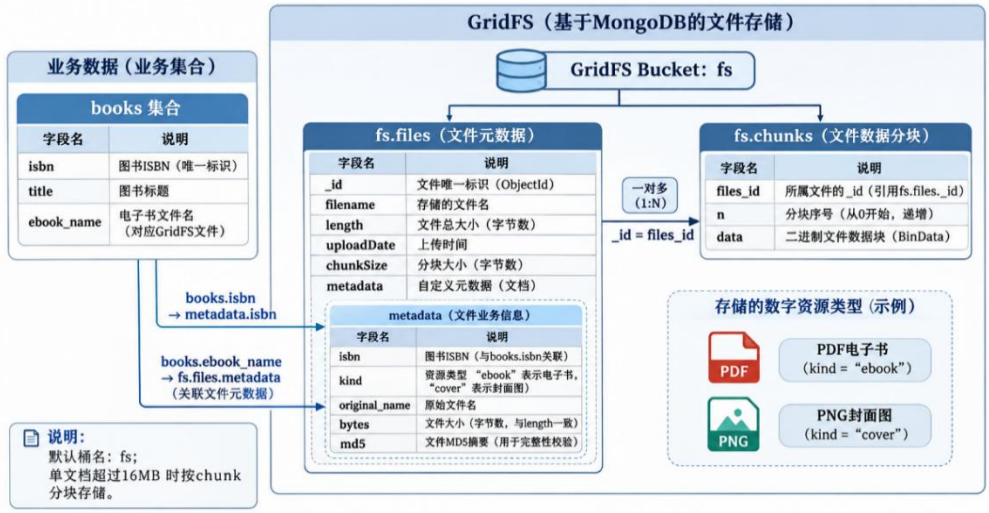


图 5-1 基于 GridFS 的数字资源存储结构图

5.2 上传实现

上传入口是 gridfs_manager.upload：先读取文件全部字节并计算 MD5，再组装 metadata，最后调用 GridFSBucket(db,bucket_name="fs").upload_from_stream 写入，返回 file_id。批量上传由 upload_manifest 完成，它读取 data/gridfs_files/files_manifest.txt，按 ebooks、covers 两组核对文件后逐个上传，并记录返回的 file_id。建库脚本首次建库时调用，fs.files 已有记录则跳过。本次共上传 6 个文件（3 本 PDF、3 张 PNG），字节数与清单记录一致。

5.3 下载与校验

下载按业务语义而不是 file_id 进行：先按书名模糊检索带 ebook_name 的图书，取出 isbn，再用 metadata.isbn 与 metadata.kind 在 fs.files 中查找，按 uploadDate 倒序取最新一条，最后用 open_download_stream(file_id)读取全部字节并写入 downloads 目录。落盘后与源文件比较字节数和 MD5，两项都一致才认为文件完好，实测 6 个文件全部通过。上传与下载走同一套接口，单机、副本集与分片环境只需改连接串。

表 5-1 GridFS 数字资源存储及完整性校验结果

文件名称	文件名称	文件名称	文件名称
类型	类型	类型	类型
文件大小（Byte）	文件大小（Byte）	文件大小（Byte）	文件大小（Byte）
完整性校验	完整性校验	完整性校验	完整性校验

ebook_1.pdf PDF 39837 通过	ebook_1.pdf PDF 39837 通过	ebook_1.pdf PDF 39837 通过	ebook_1.pdf PDF 39837 通过

6 架构演进：副本集部署与故障转移、分片集群与分片键论证

6.1 演进路线

平台按“单机—副本集—分片集群”三步演进，三种形态以不同端口并存：单机 27017、副本集 rs027018、分片集群 27019—27022。应用只通过 mongodb_connect.py 读取 MONGO_URI 等环境变量，换环境只改连接串，业务代码不动。

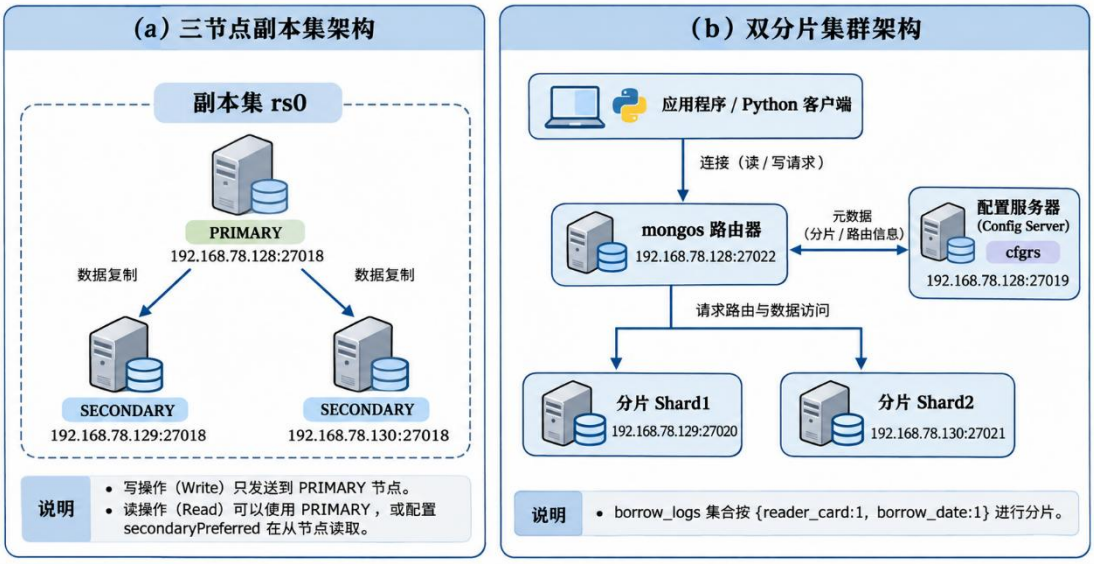


图 6-1 MongoDB 分布式部署架构图

6.2 副本集部署与故障转移

副本集 rs0 由 192.168.78.128、129、130 组成，统一端口 27018、数据目录 /data/rs0。三台统一版本后用 openssl 生成同一份 keyfile（权限 400），分别以--replSetrs0--keyFile 启动，在 128 上执行 rs.initiate，优先级设为 2、1、1，约十秒后一台 PRIMARY、两台 SECONDARY。认证先行：admin 库建 root，smartlib 库建 app_rw 与 app_ro；导入仍由 setup_smartlib.py 完成，只把连接串换成带 replicaSet=rs0 的三节点地址，导入后 50/30/3000 与 6 个文件与单机一致。

默认读偏好 primary 只读主节点，从节点须显式指定 readPreference=secondary 才能读；向主节点写入 TEST-RS-001 后从节点能读到，说明 oplog 复制正常。primary 适合借阅后立即查询，secondaryPreferred 适合馆藏浏览与报表，以秒级延迟换读吞吐。故障转移用 rs.stepDown(60)：原主 60 秒内不参与选举，剩余两台 15 秒选出新主；退位期间写原主返回 NotWritablePrimary，客户端需重试，走 secondaryPreferred 的读不受影响。原主 60 秒内以 SECONDARY 回归，冻结期结

束后因优先级更高仍可能重新当选。节点增删用 rs.add 与 rs.remove，成员数由 4 恢复为 3。

表 6-1 副本集主节点切换实验结果

实验阶段	192.168.78.128	192.168.78.129	192.168.78.130
切换前	PRIMARY	SECONDARY	SECONDARY
切换后	SECONDARY	PRIMARY	SECONDARY

执行 rs.stepDown(60)后，原主节点主动退位，副本集重新进行选举，约 15 秒后 192.168.78.129 成为新的 PRIMARY 节点，其余两个节点保持 SECONDARY 状态，验证了副本集的自动选举和主节点切换机制。

6.3 分片集群部署

分片集群按“配置服务器—分片—路由”搭建：cfgrs 为单节点配置副本集（27019），shard1、shard2 位于 129、130（27020、27021），mongos 在 128（27022），应用只连 mongos。教学环境每个分片是单节点，生产应为副本集。启动顺序为 cfgrs、shardsvr、mongos；启用 keyfile 时先在 admin 库建 root，再 sh.addShard、创建业务账号并 sh.enableSharding("smartlib")。

分片前必须先建索引：borrow_logs 建{reader_card:1,borrow_date:1}复合索引，再用同一字段 shardCollection；分片键必须是索引前缀且字段顺序一致，否则报 couldnotfindindex。索引就绪后再导入 3000 条流水。

表 6-2 双分片集群数据分布及查询路由结果

验证项目	实验结果
分片数量	2 个
borrow_logs 总记录数	3000 条
shard1 记录数	1500 条
shard2 记录数	1500 条
Chunk 数量	2 个
分片键	reader_card + borrow_date

表 6-3 定向查询与跨分片查询对比

查询类型	查询条件	执行阶段	参与分片	返回记录
定向查询	reader_card + borrow_date	SINGLE_SHARD	1	66
跨分片查询	status=逾期	SHARD_MERGE	2	429

6.4 数据分布与查询路由

3000 条流水约 600KB，远小于 128MB 的 chunk 上限，导入后只有一个 chunk，均衡器无块可迁，数据全落一片。取按分片键排序的第 1501 条作为切分点，splitAt 后 moveChunk 到 shard2，两片各约 1500 条；也可在空集合上先切分迁移再导入，v6_presplit.js 即此顺序。

带完整分片键的查询命中 SINGLE_SHARD，只访问一个分片；不带分片键的（如 status 为逾期）走 SHARD_MERGE，两片参与并归并。分片键既决定数据怎么放，也决定查询怎么走。

表 6-4 不同分片键方案对比

分片键方案	分片键方案	分片键方案
主要优点 主要不足	主要优点 主要不足	主要优点 主要不足
{reader_card:1,borrow_date:1} 支持读者及时间范围查询，具备较好的组合区分能力 不利于按状态直接定向查询	{reader_card:1,borrow_date:1} 支持读者及时间范围查询，具备较好的组合区分能力 不利于按状态直接定向查询	{reader_card:1,borrow_date:1} 支持读者及时间范围查询，具备较好的组合区分能力 不利于按状态直接定向查询

6.5 分片键论证

6.5 分片键论证

本项目选择 {reader_card: 1, borrow_date: 1} 作为 borrow_logs 集合的复合分片键，主要从数据基数、写入分布、查询模式及索引复用四个方面进行综合考虑。

首先，从数据基数与区分能力来看，本项目提供的借阅流水共 3000 条，涉及 30 名读者，数据时间范围覆盖 2023—2025 年。若仅采用 reader_card 作为分片键，其取值数量仅为 30，区分度相对有限，可能导致不同读者对应的数据分布粒度不足。将 borrow_date 作为第二个分片键字段后，可以进一步利用不同借阅时间对数据进行细粒度划分，使复合键具有更高的区分能力。相比之下，status 字段仅包含已归还、借出和逾期三种状态，基数较低，不适合作为主要分片依据。因此，采用读者标识与借阅时间相结合的复合分片键，更符合本项目借阅流水的数据组织特点。

其次，从数据分布与写入负载来看，借阅流水在不同读者之间具有一定的分散性，每位读者约有 100 条借阅记录。复合分片键通过读者标识和借阅时间共同确定数据所在的键值范围，能够避免仅使用低基数字段造成的分布粒度不足。同时，borrow_date 作为第二个字段，可以进一步区分同一读者在不同时间产生的借阅记录，有利于数据范围划分和后续扩展。需要说明的是，复合分片键并不意味着数据能够在所有情况下自动实现均匀分布，其实际效果还受到数据规模、键值分布及 Chunk 划分情况的影响。本项目原始 3000 条借阅流水总量约为 600 KB，远小于 MongoDB 默认 128 MB 的 Chunk 分裂阈值，因此初始数据导入后未能自动形成两个分片的数据分布。针对这一情况，本项目通过 splitAt 和 moveChunk 操作对数据范围进行人工划分与迁移，最终实现两个分片各承载约 1500 条记录，验证了双分片环境下的数据分布能力。

再次，从业务查询需求来看，借阅流水的典型查询场景包括指定读者在某一时间范围内的借阅记录查询，以及面向全体读者的借阅状态统计。其中，前一种查询同时使用 reader_card 和 borrow_date 作为筛选条件，与复合分片键的字段组成及顺序保持一致，能够充分利用分片键进行查询路由，减少不必要的分片访问。本项目通过实际实验验证，包含完整分片键条件的查询采用 SINGLE_SHARD 执行，仅访问一个分片，最终返回 66 条记录。相比之下，以

status=逾期 为条件的查询未包含分片键字段，无法根据分片键直接确定数据所在位置，需要向两个分片发送查询请求，并通过 SHARD_MERGE 完成结果归并，最终返回 429 条记录。两种查询结果表明，合理选择分片键不仅影响数据存储位置，也直接决定查询路由方式及跨分片访问开销。

此外，从索引复用与系统维护角度来看，本项目在执行分片操作前建立了 {reader_card: 1, borrow_date: 1} 复合索引，并将其作为分片键所需的索引基础。该索引既能够支持读者及借阅时间范围查询，也能够满足复合分片键的索引要求，避免针对相同字段重复建立相似索引，从而减少索引维护和存储开销，提高数据库结构的一致性。

为进一步验证方案的合理性，本项目对其他候选分片键进行了比较。若选择 {borrow_date: 1}，虽然能够支持借阅时间范围查询，但借阅时间具有持续递增的特点，可能使新增数据集中进入最后一个键值范围，形成写入热点。若选择 {status: 1}，由于字段取值仅有三种，容易造成不同分片之间的数据分布不均。若选择 {book_isbn: "hashed"}，虽然哈希分布有利于提高数据分散程度，但不利于按照读者和借阅时间开展范围查询。若选择 {reader_card: "hashed"}，虽然可以改善不同读者之间的数据分布，但会损失读者标识的有序性，不利于相关范围查询的定向路由。综合比较，上述方案均未能同时兼顾本项目的主要查询需求与数据分布要求。

综上所述，{reader_card: 1, borrow_date: 1} 复合分片键能够较好地兼顾借阅流水的数据区分能力、读者及时间范围查询需求和索引复用要求，是当前业务场景下较为合适的选择。但该方案也存在一定局限性，例如按借阅状态或学院进行统计时，由于查询条件不包含分片键，仍需要访问多个分片并进行结果归并。随着数据规模和业务访问模式不断变化，后续仍需结合实际数据分布、查询负载及分片热点情况，对分片策略进行持续评估与优化。

表 6-5 主节点故障转移及业务影响

实验阶段	实验阶段	实验阶段	实验阶段
主节点状态	主节点状态	主节点状态	主节点状态
读操作	读操作	读操作	读操作
写操作	写操作	写操作	写操作
故障前	故障前	故障前	故障前

7 问题与总结

7.1 技术难点排查

难点一：分片集合创建失败，报 couldnotfindindex。最初先分片后补索引，mongos 直接拒绝；排查发现 borrow_logs 只有单字段索引，而分片键是 {reader_card:1,borrow_date:1}，字段与顺序都不匹配。改为先建复合索引再分片，并把“先索引、后分片”写进部署脚本，问题不再复现。

难点二：3000 条数据全落在一个分片。getShardDistribution()显示 shard1 有 3000 条、shard2 为 0；查看 config.chunks 只有一个 chunk，原因是 600KB 数据远小于 128MB 的 chunk 上限，均衡器不会主动切分。改为手动 splitAt 第 1501 条记录的键值，再 moveChunk 到 shard2，两片各 1500 条。

难点三：副本集下从节点读不到刚写入的数据。直连从节点查询返回空，rs.status()却显示三节点健康。问题在连接串：缺少 replicaSet=rs0 与 readPreference，直连从节点时因不是主节点而拒绝读。补上 replicaSet、authSource 与 readPreference=secondaryPreferred 后正常；退位期间写请求返回 NotWritablePrimary 属预期行为，客户端重试即可。

表 7-1 项目实施过程中的关键技术问题及解决方案

技术问题	原因分析	解决措施	验证结果
分片失败	缺少符合分片键要求的复合索引	先建立复合索引，再执行分片	分片成功
数据集中在单个分片	数据量小于 Chunk 默认上限	手动 splitAt 和 moveChunk	两片各 1500 条
从节点读取失败	连接参数缺少副本集及读取偏好配置	完善 replicaSet、authSource 和 readPreference	从节点读取正常

7.2 项目反思

最大的收获是“部署形态”与“应用接口”可以解耦：三套环境共用一套代码，切换只改连接串；但应用对拓扑仍无感知，故障转移中的写重试与读偏好需要显式配置，不能指望驱动替业务做决定。

分片键是一次性决策。基数、写分布、查询模式互相牵制，按“读者加时间”复合分片兼顾了均匀写入与定向查询，却牺牲了按状态、按学院查询的局部性；数据量小时均衡器不工作，“部署成功”不等于“效果可见”，必须用 splitAt、moveChunk 和 explain 主动验证。

副本集提升可用性也有代价：选举期间存在秒级写不可用，读偏好是在一致性与吞吐之间取舍。不足是分片均为单节点、跨分片事务尚未验证、GridFS 未按 ISBN 去重、固定集合的审计日志会淘汰旧记录；后续可把每个分片改为副本集，并补充监控、重试与一键部署。

表 7-2 项目核心功能验收结果

验收项目	目标要求	实际结果	状态
图书数据	50 条	50 条	通过
读者数据	30 条	30 条	通过
借阅流水	3000 条	3000 条	通过
GridFS 文件	6 个	6 个，字节及 MD5 一致	通过
副本集	3 节点	1 主 2 从，完成选举验证	通过
分片集群	2 个分片	1500 条/1500 条	通过
查询路由	定向及跨分片查询	SINGLE_SHARD、SHARD_MERGE	通过

综上，本项目完成了智慧图书馆数字资源平台的数据库设计、核心功能开发及分布式架构部署，实现了图书与读者信息管理、借阅检索、聚合统计、索引优化、权限控制和 GridFS 文件存储等功能，并通过三节点副本集和双分片集群实

验验证了 MongoDB 的数据复制、故障选举及分片查询路由机制。各项核心功能均达到预期验收目标，初步形成了集数据管理、业务应用与分布式部署于一体的数据库应用平台。通过本次项目实践，进一步掌握了 MongoDB 文档数据建模、聚合查询、性能优化及分布式架构配置方法，为后续平台功能扩展与性能优化奠定了基础。

附录　过程记录截图

本附录按任务编号汇总项目实施过程中的过程记录截图，共 46 张，涵盖 T3 单机实现、T4 应用开发、T5 副本集、T6 分片集群与 T7 验收五个阶段。

附录 A　T3 单机实现

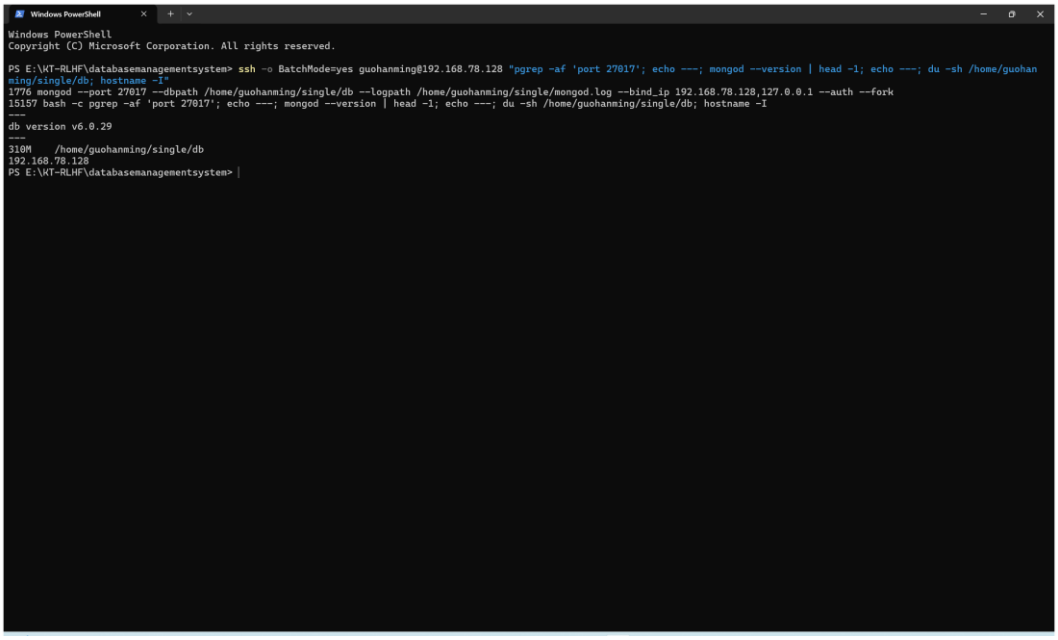


图 A-1　单机实例与数据目录

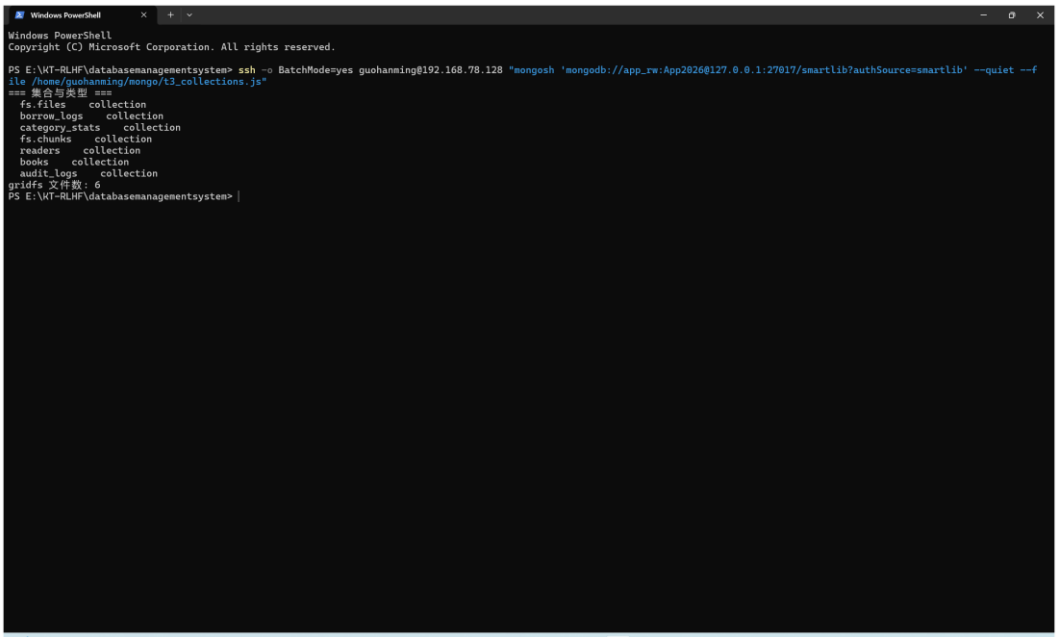


图 A-2　集合与类型

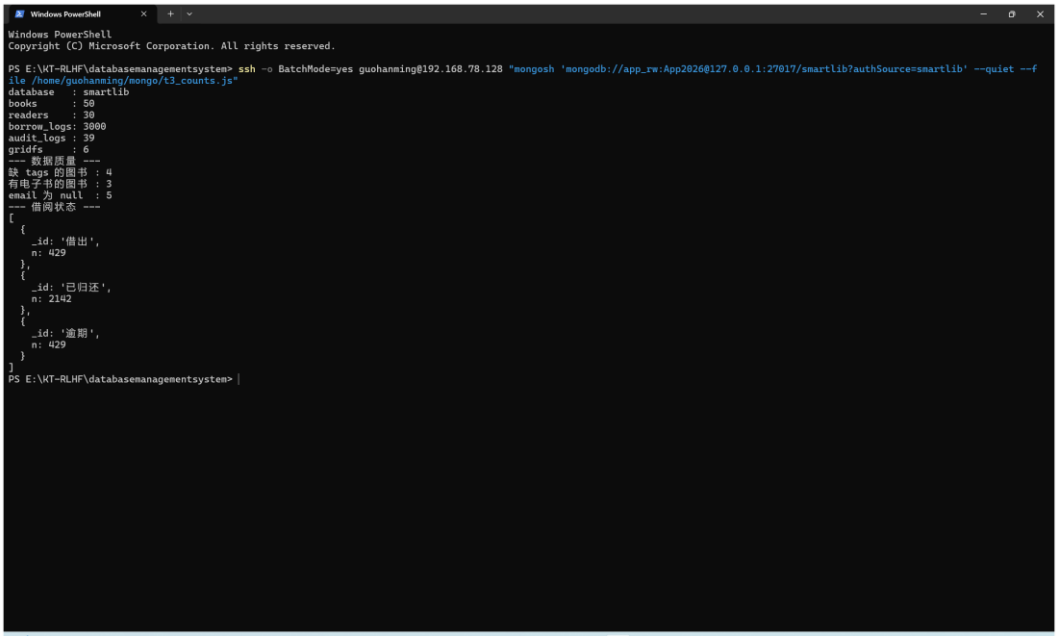


图 A-3　数量核对

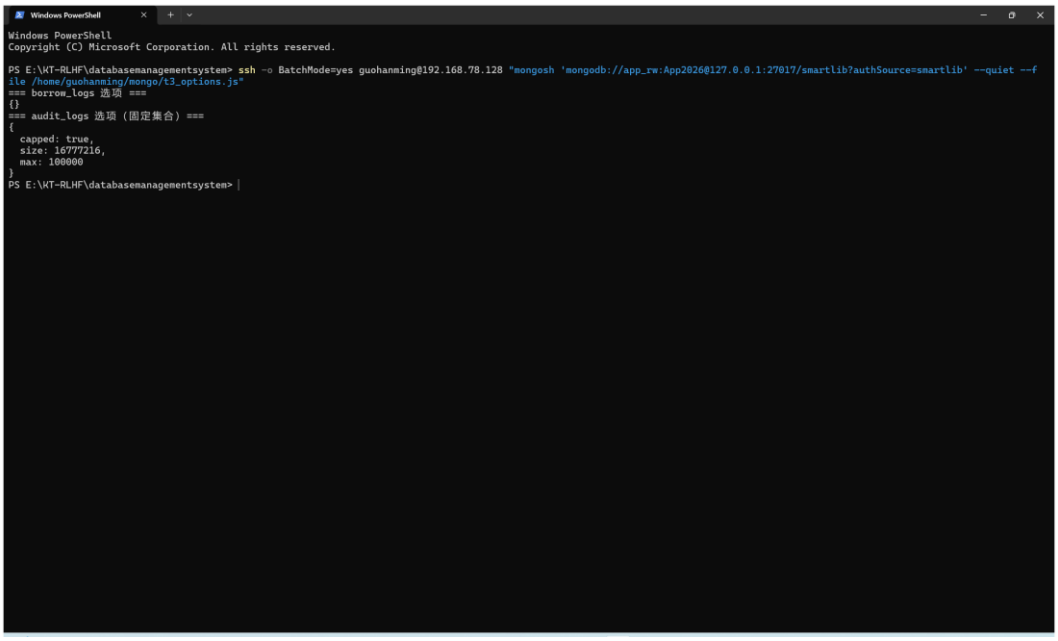


图 A-4　集合选项（audit_logs 为 capped）

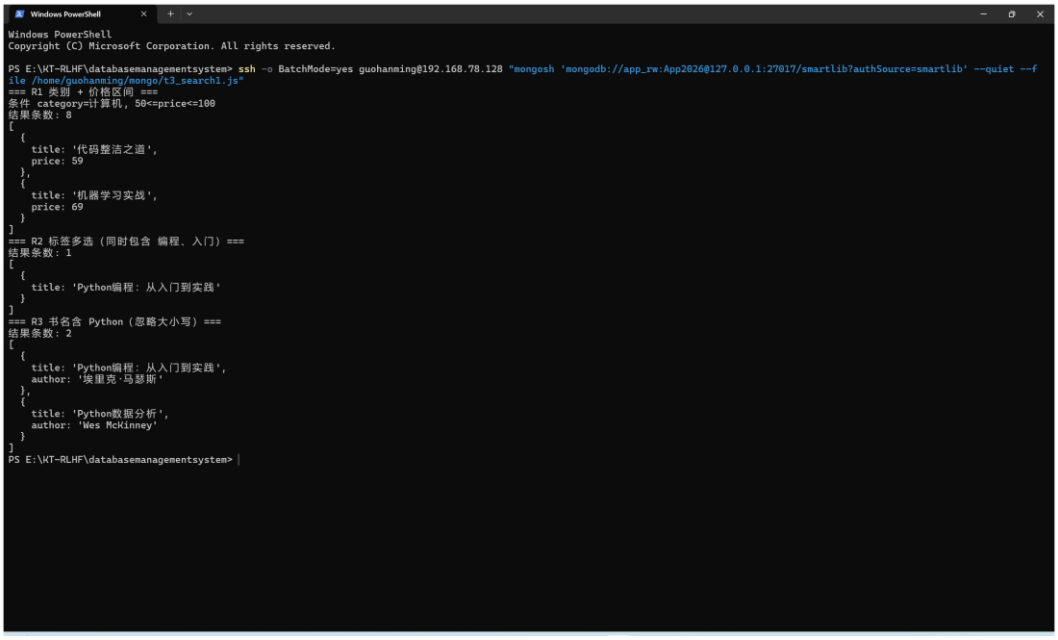


图 A-5　检索 R1-R3

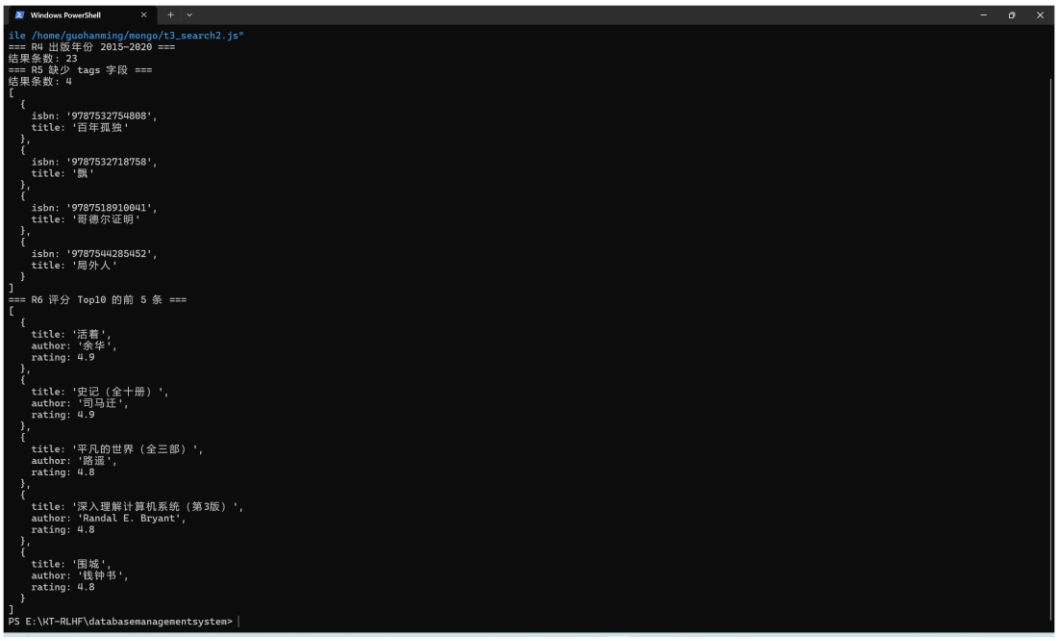


图 A-6　检索 R4-R6



图 A-7　统计 S1-S2

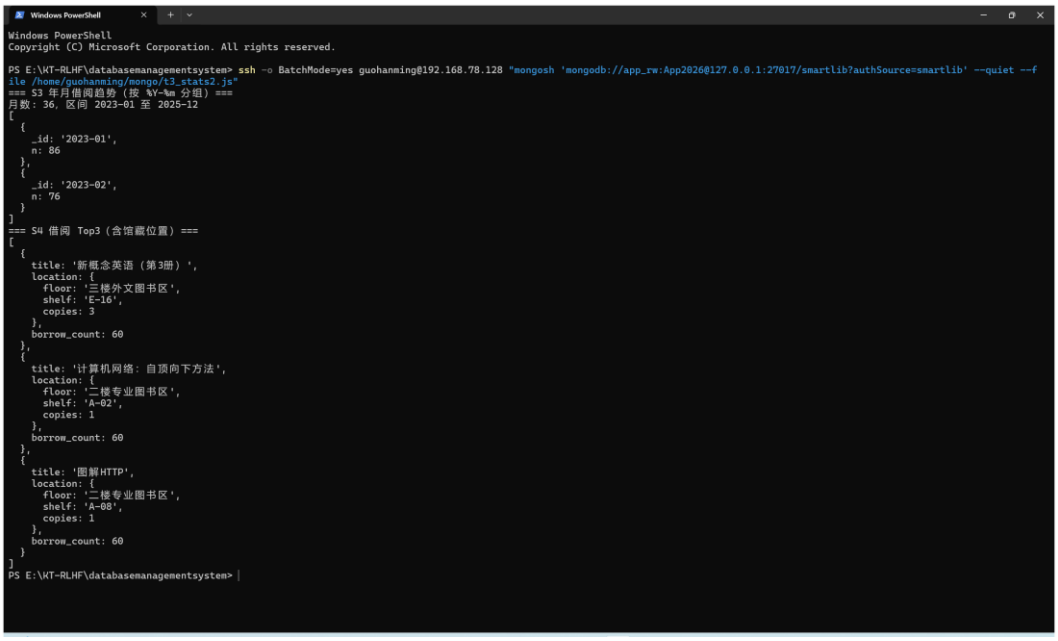


图 A-8　统计 S3-S4

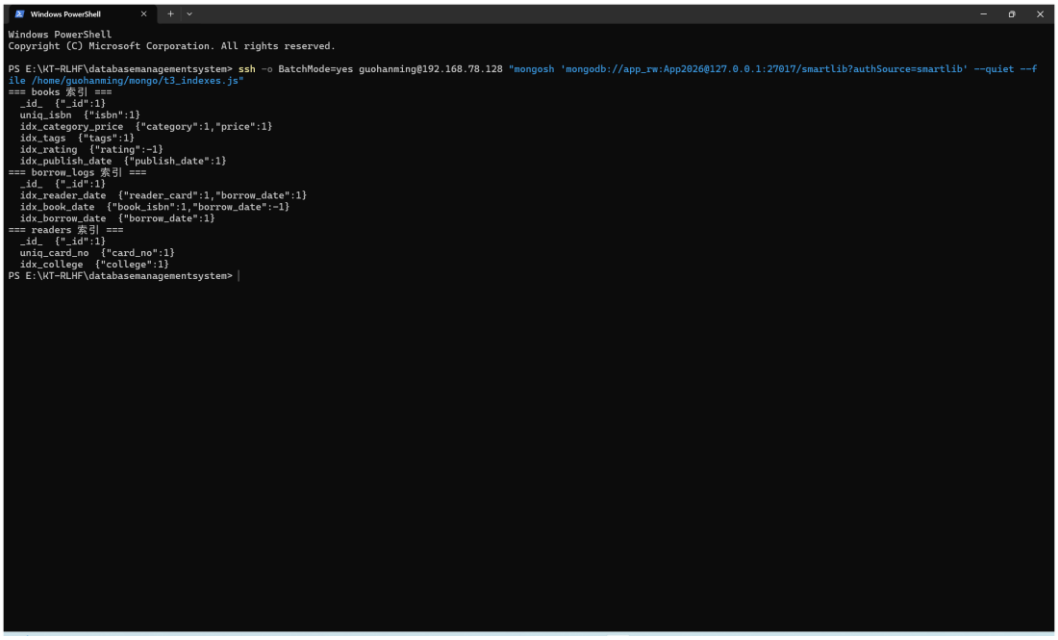


图 A-9　索引清单

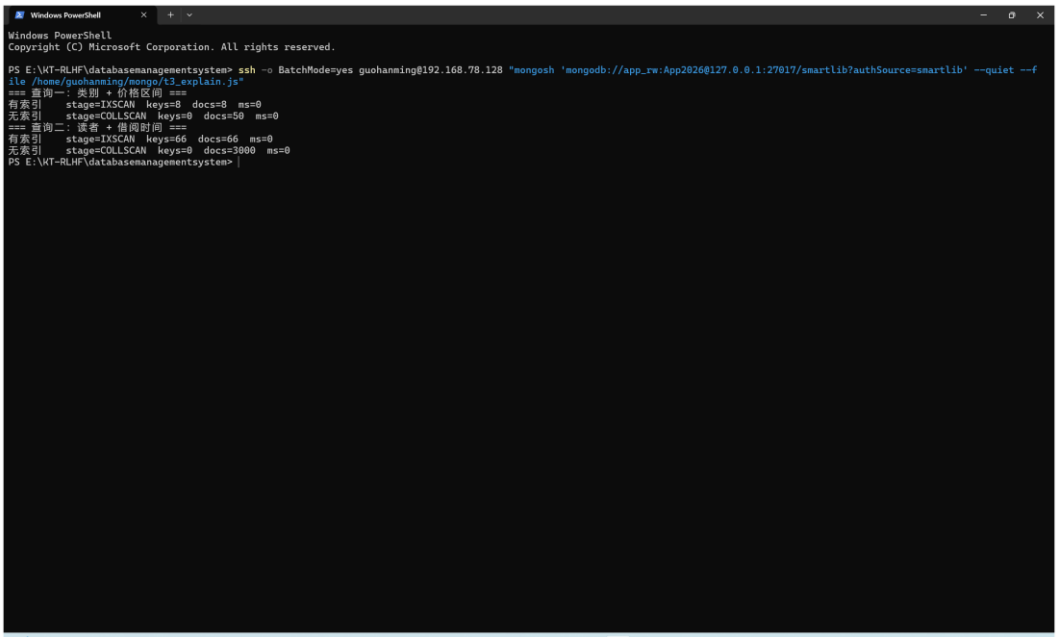


图 A-10　explain 执行计划对比

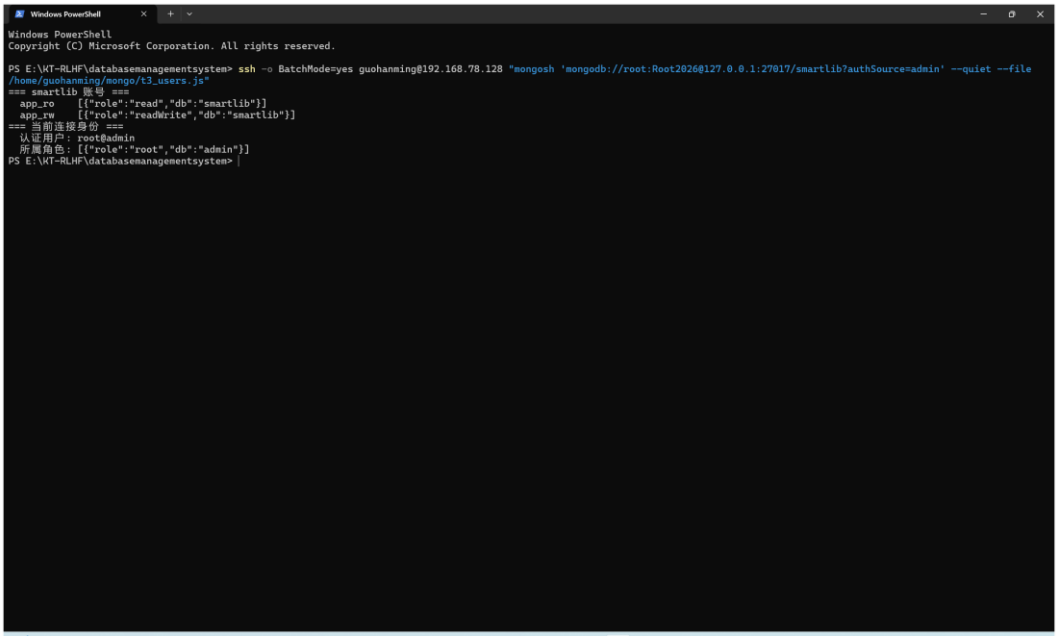


图 A-11　账号与角色

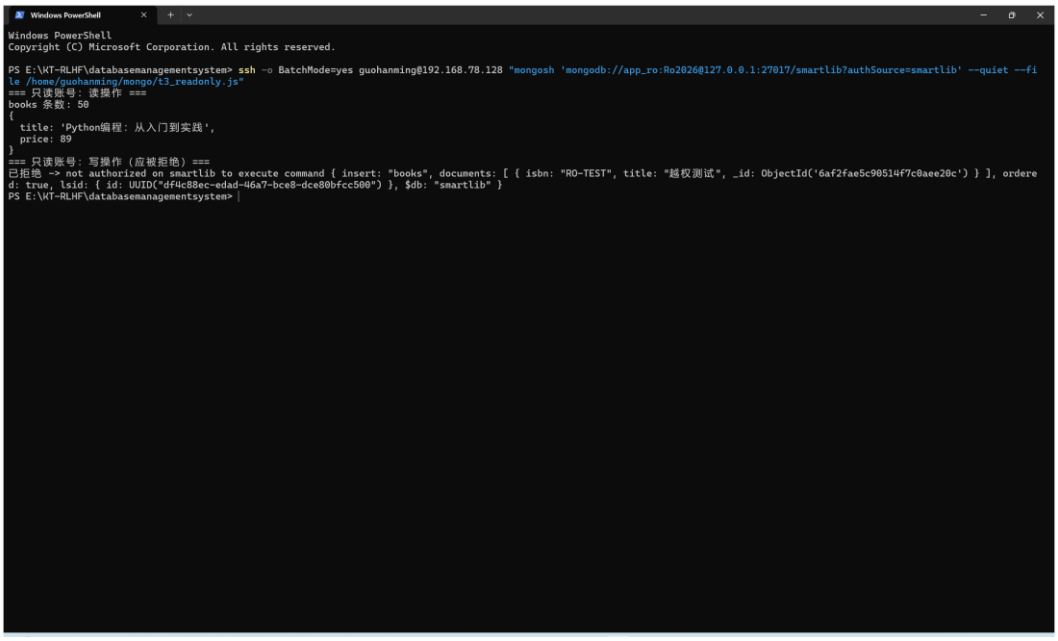


图 A-12　只读账号越权被拒

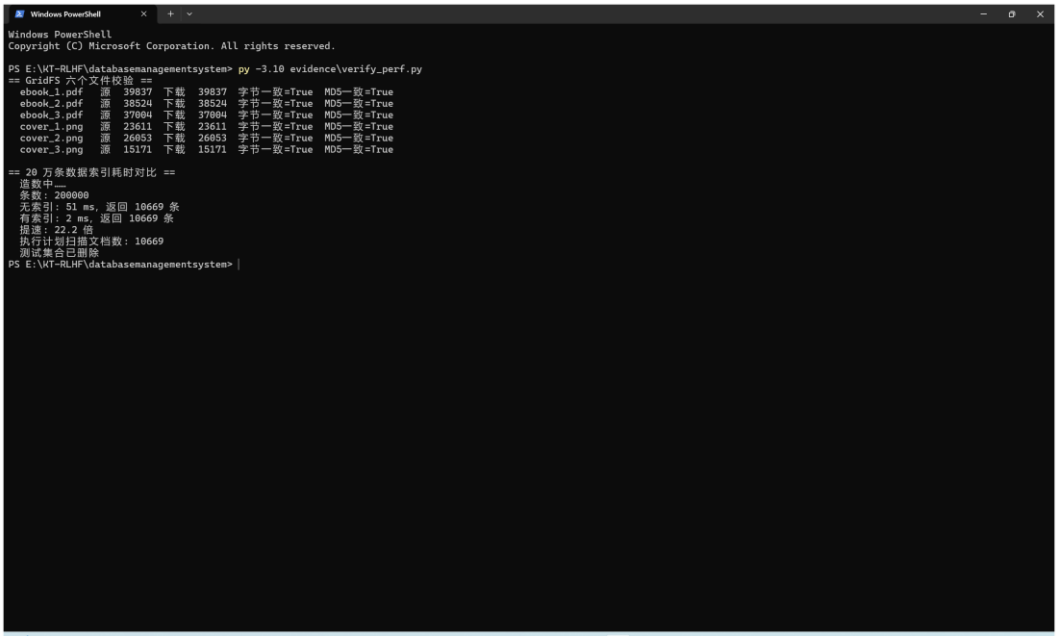


图 A-13　索引耗时对比与 GridFS 文件校验

附录 B　T4 应用开发

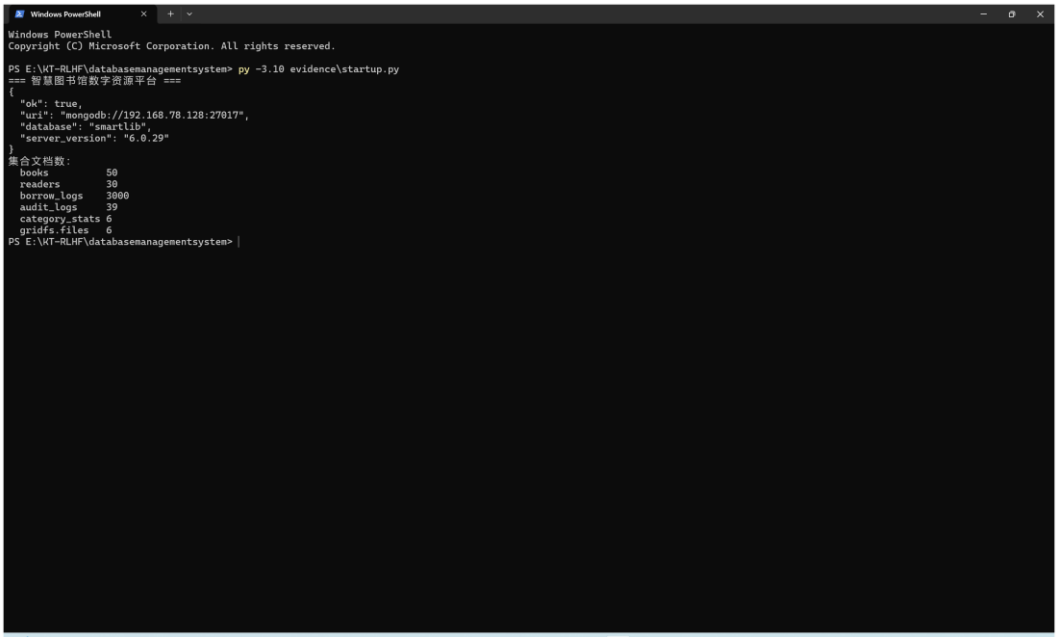


图 B-1　程序启动信息

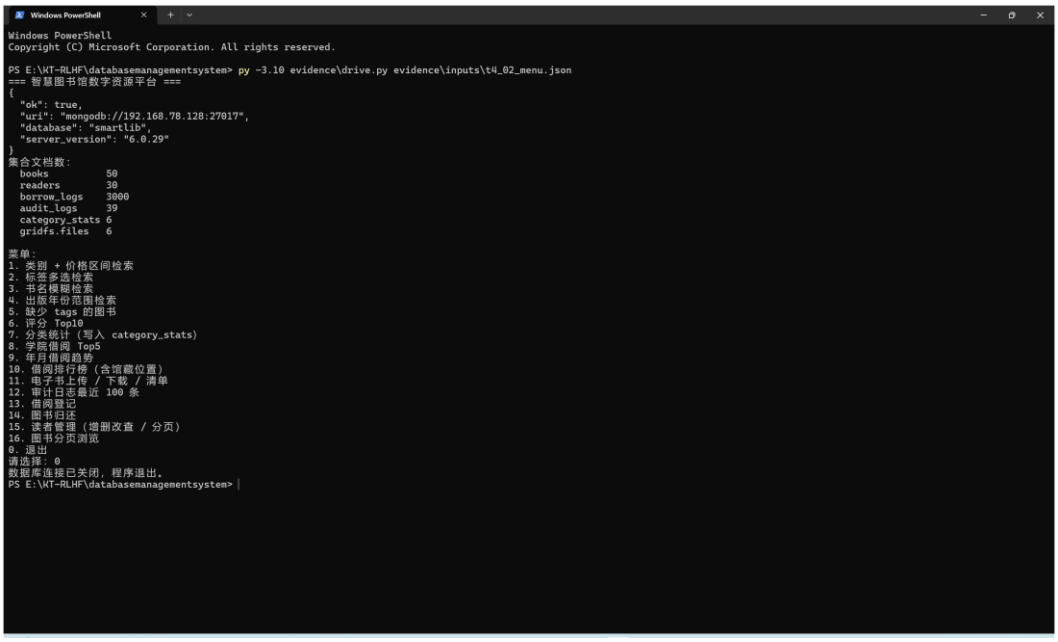


图 B-2　主菜单（16 项）



图 B-3　检索与分类统计



图 B-4　电子书上传



图 B-5　下载与字节校验

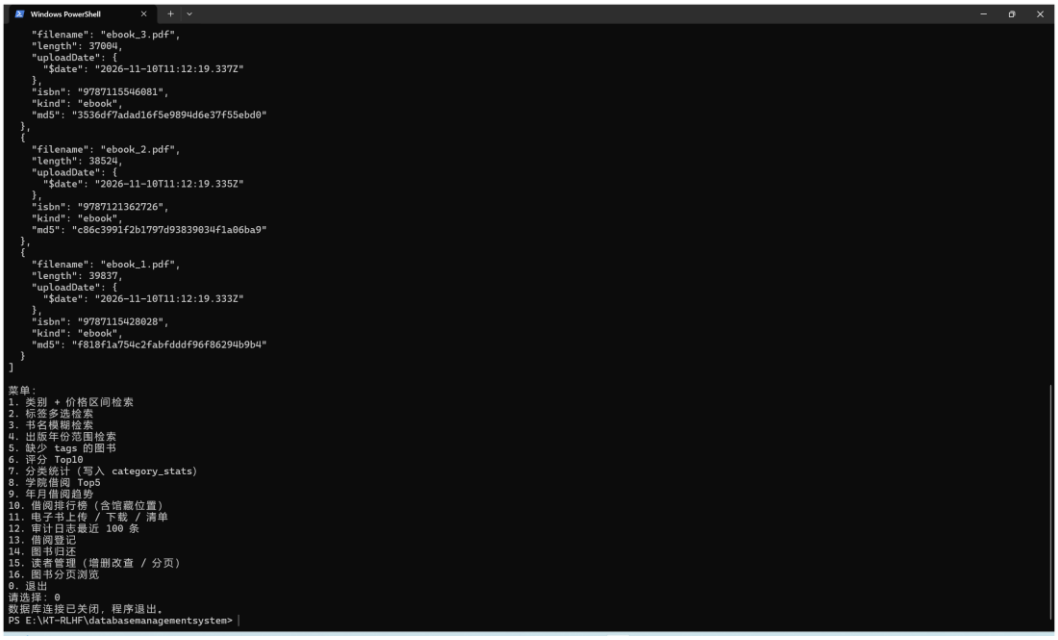


图 B-6　GridFS 文件清单



图 B-7　借阅与归还

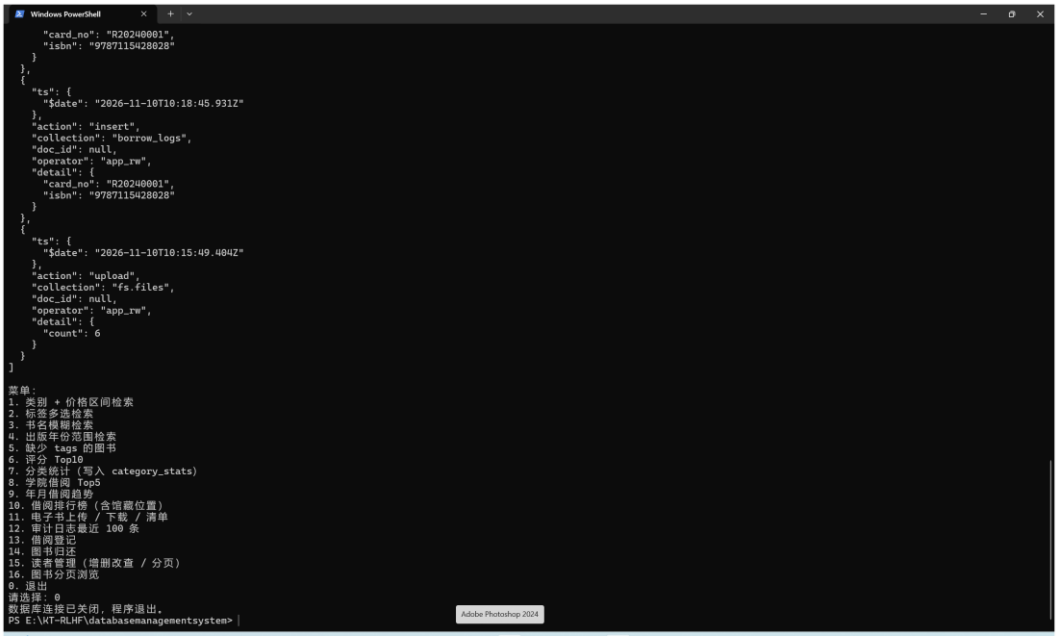


图 B-8　审计日志



图 B-9　读者管理-新增



图 B-10　读者管理-分页

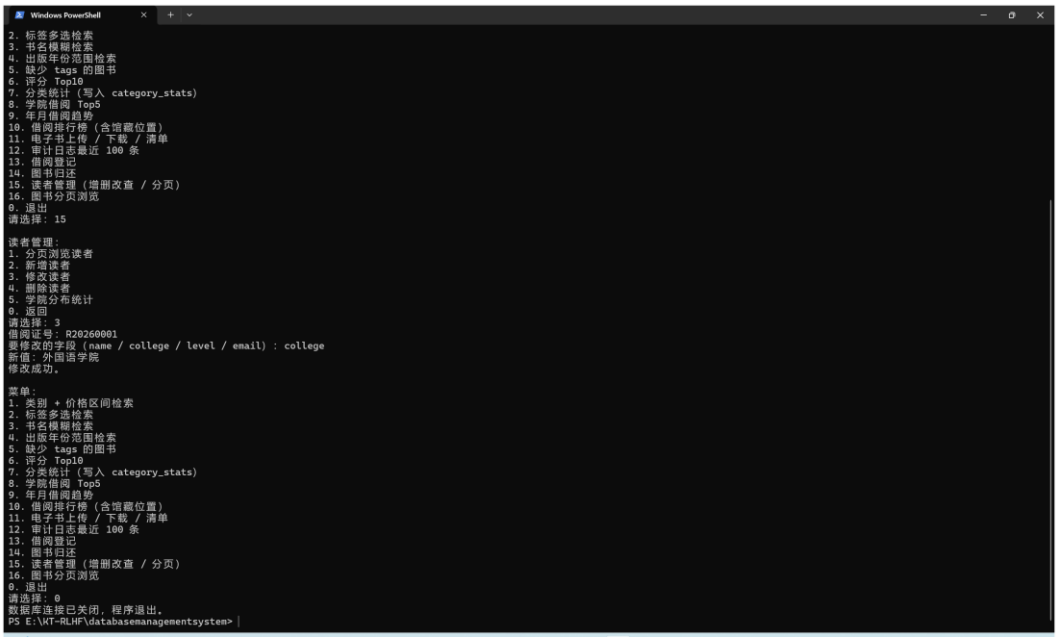


图 B-11　读者管理-修改



图 B-12　读者管理-删除

附录 C　T5 副本集

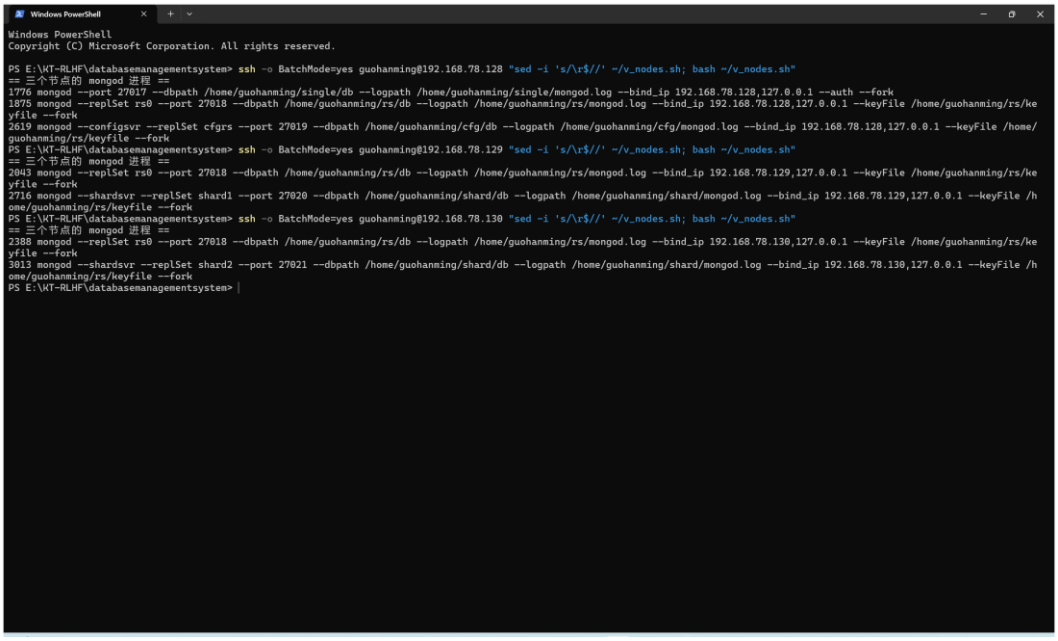


图 C-1　三个节点进程

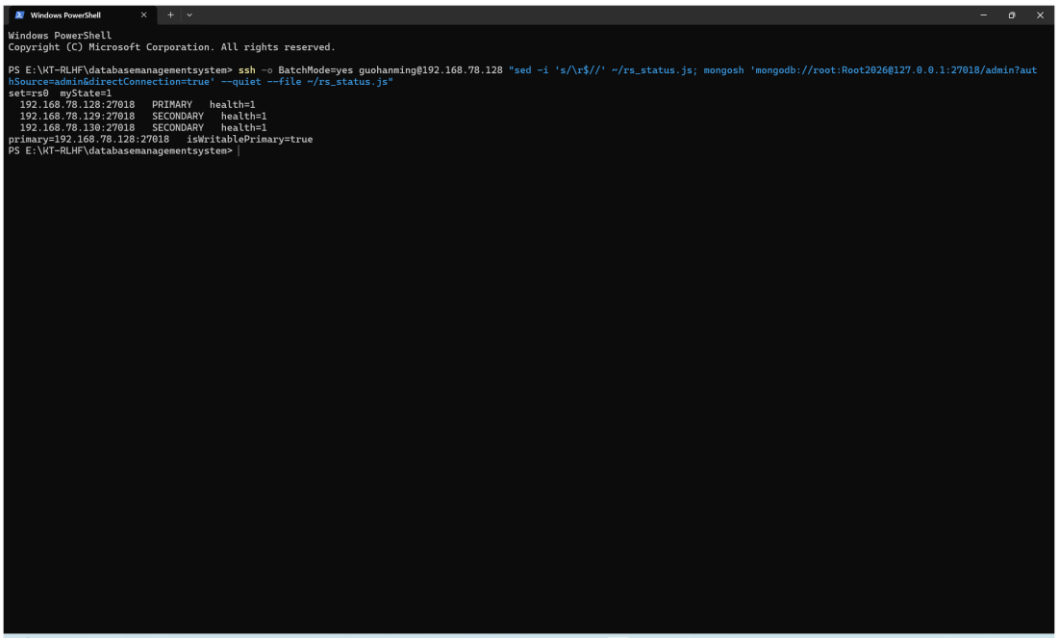


图 C-2　副本集状态

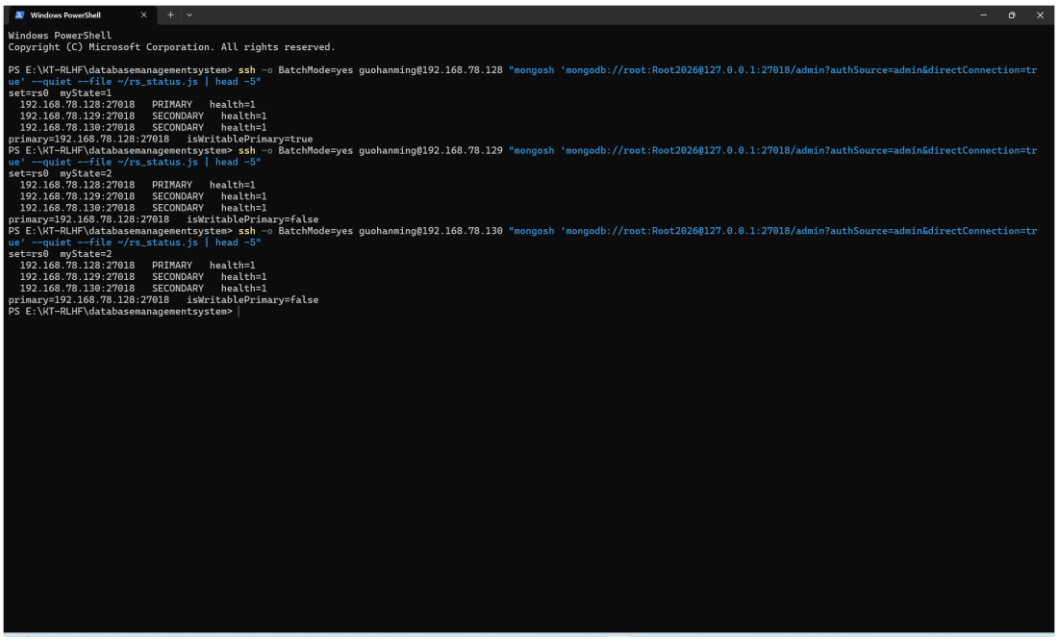


图 C-3　三节点角色

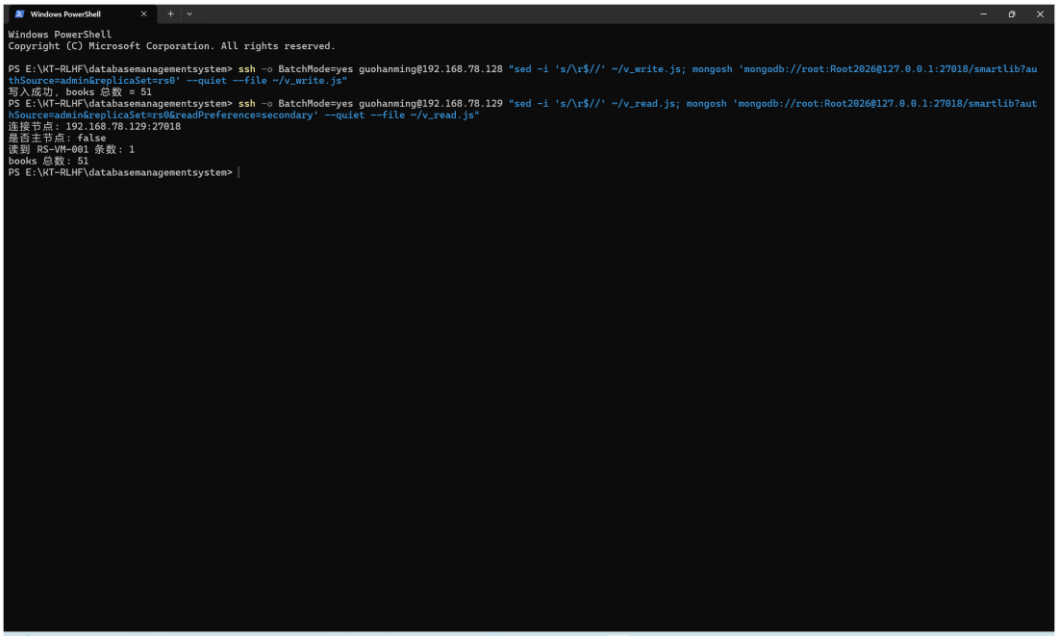


图 C-4　主写从读

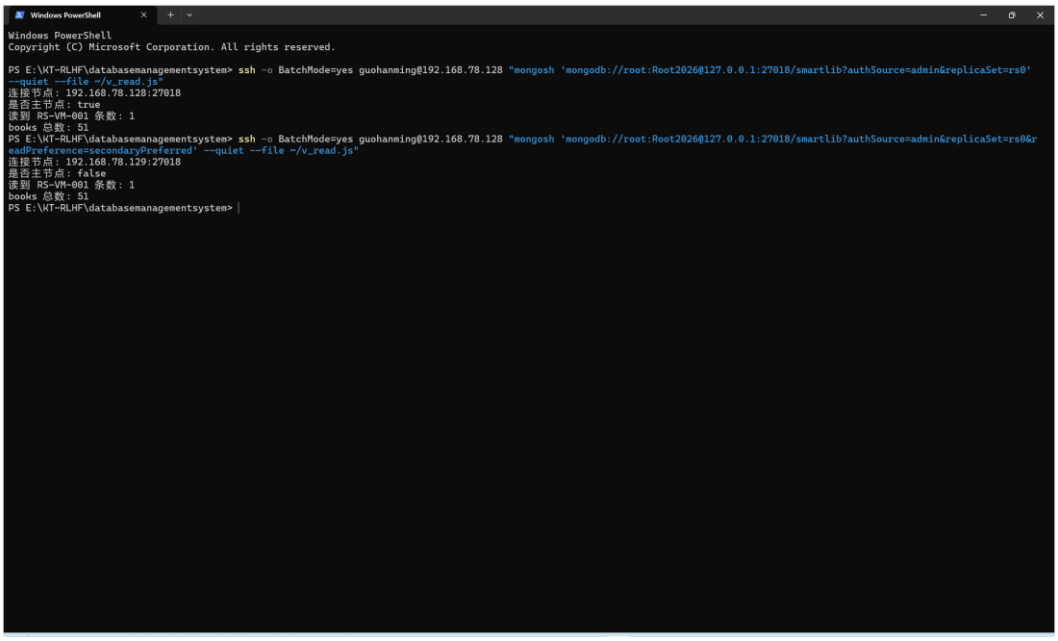


图 C-5　读偏好对比

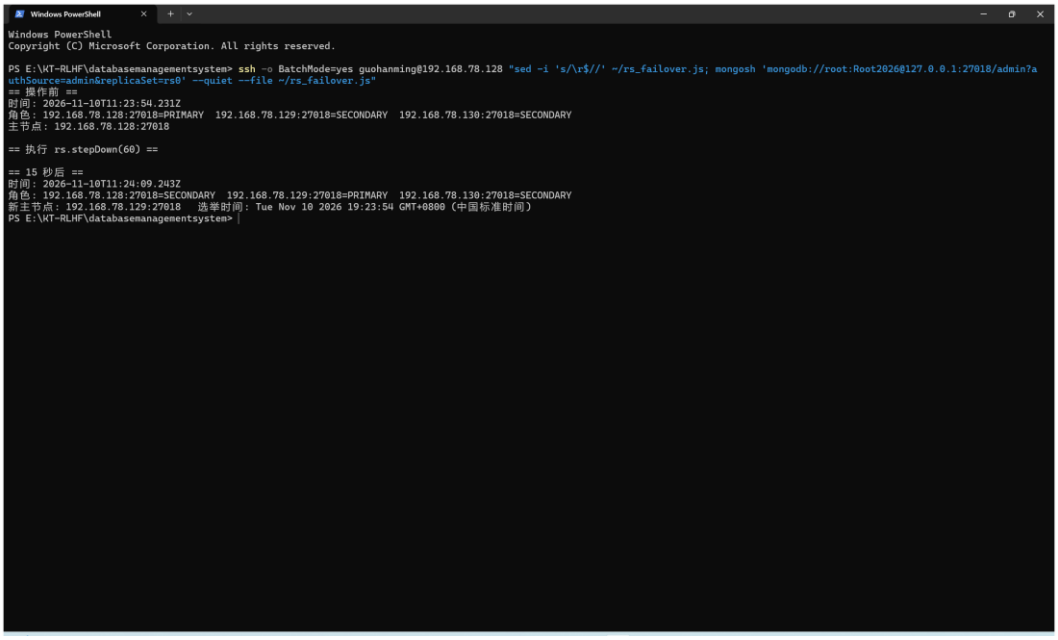


图 C-6　故障转移演练

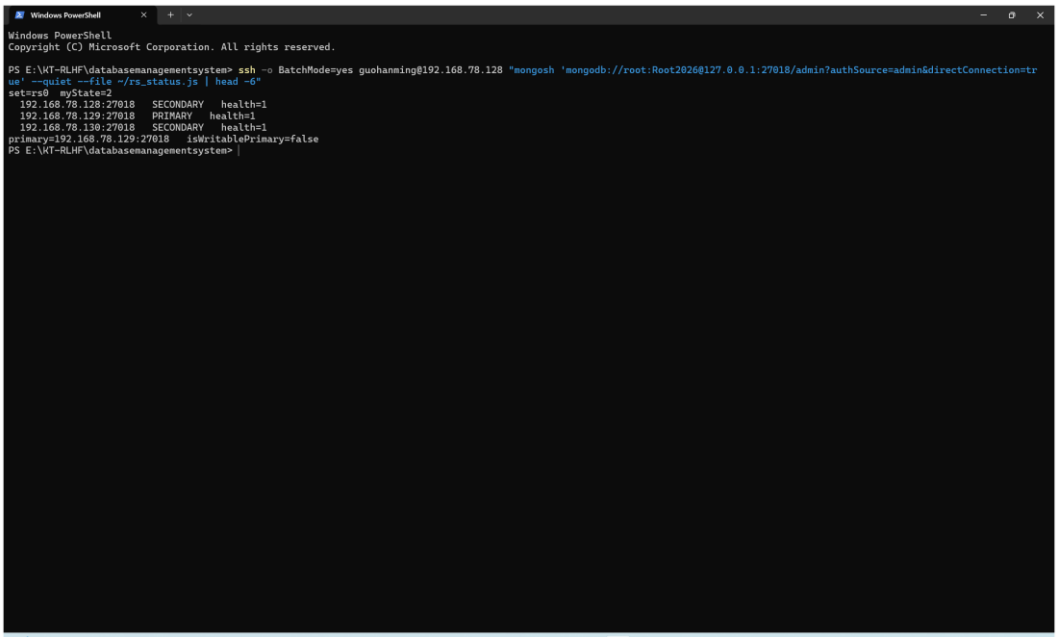


图 C-7　原主节点回归

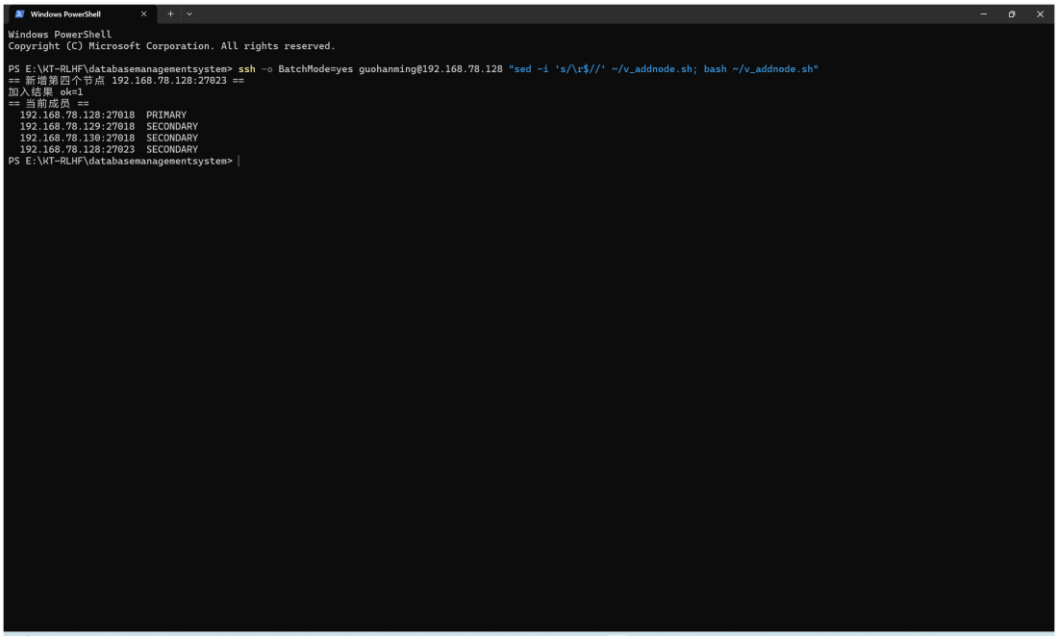


图 C-8　新增节点

附录 D　T6 分片集群

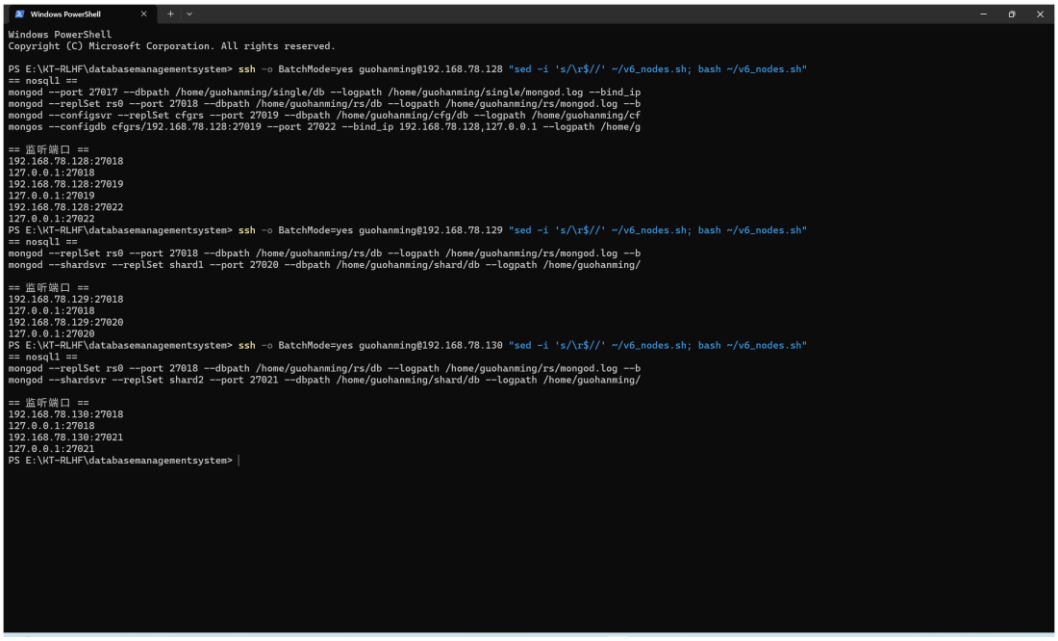


图 D-1　集群进程与端口

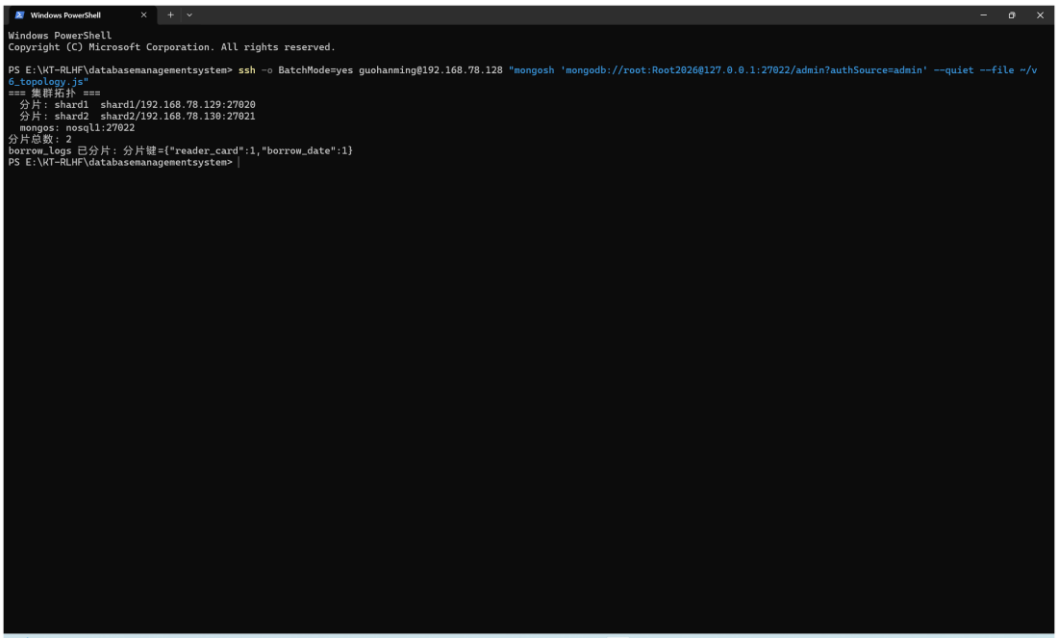


图 D-2　两个分片已加入

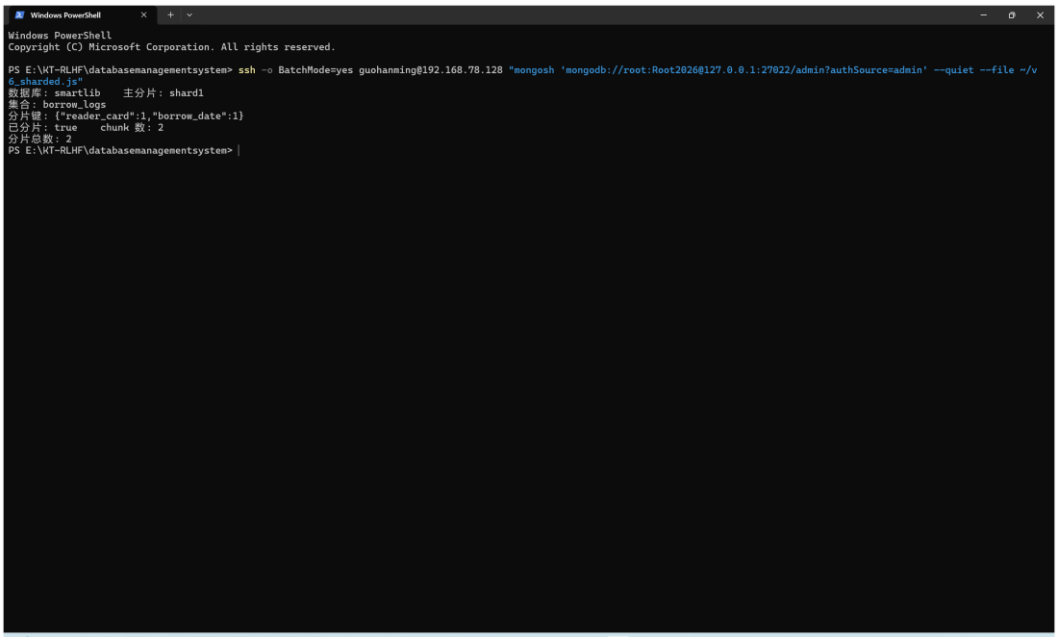


图 D-3 启用分片与分片键

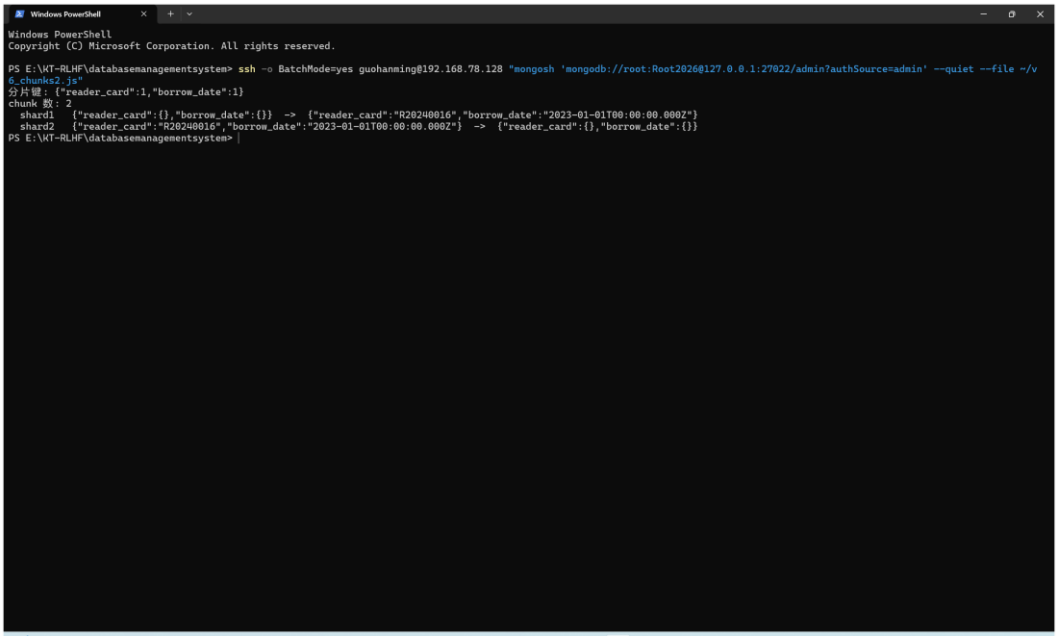


图 D-4 chunk 分布

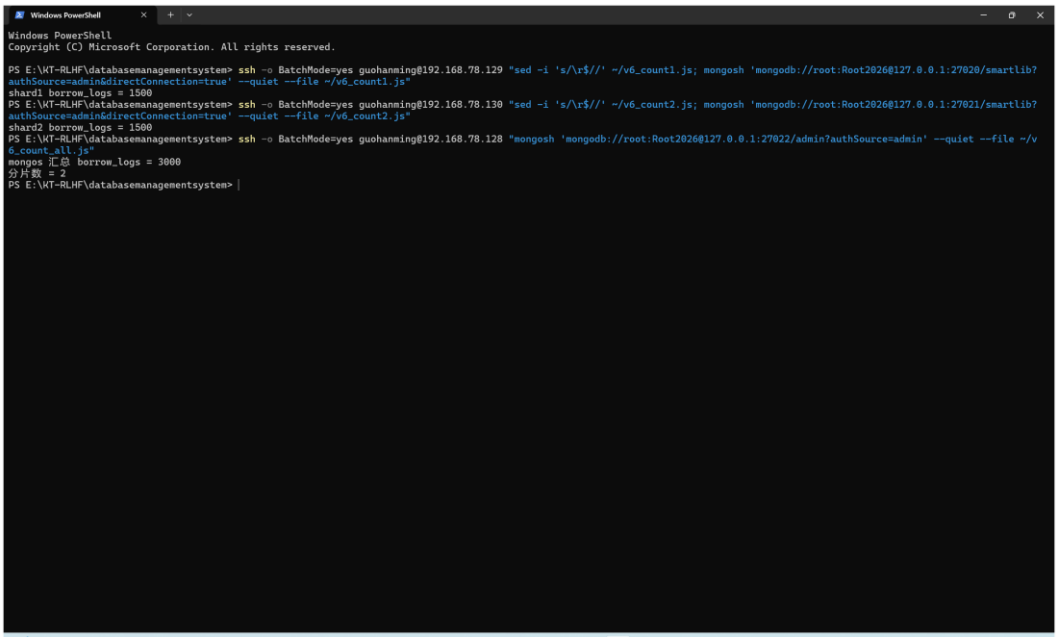


图 D-5　数据分布（两分片各 1500）

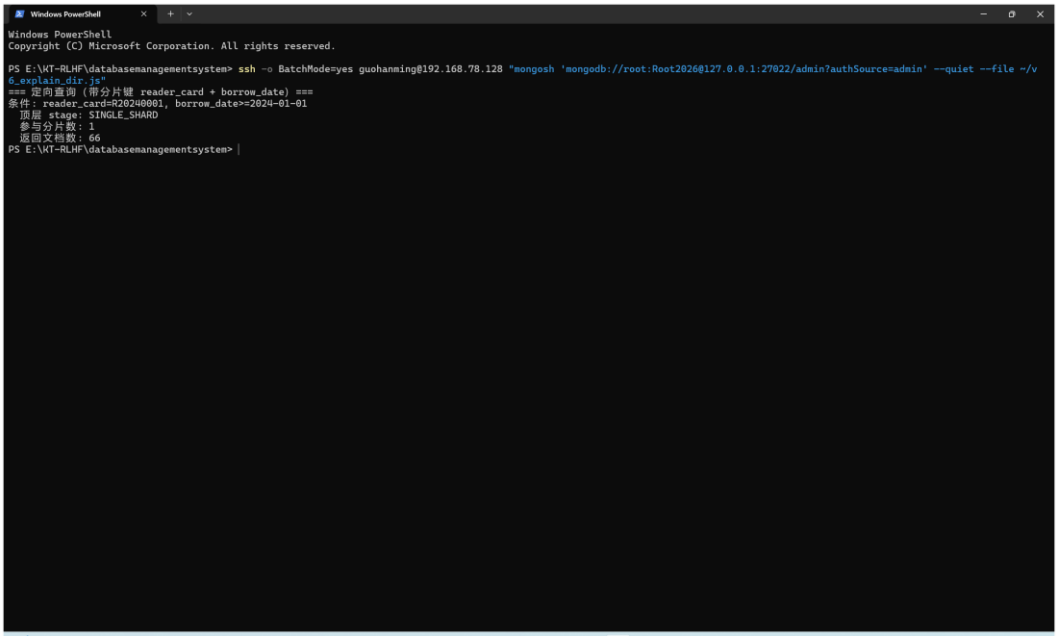


图 D-6　定向查询

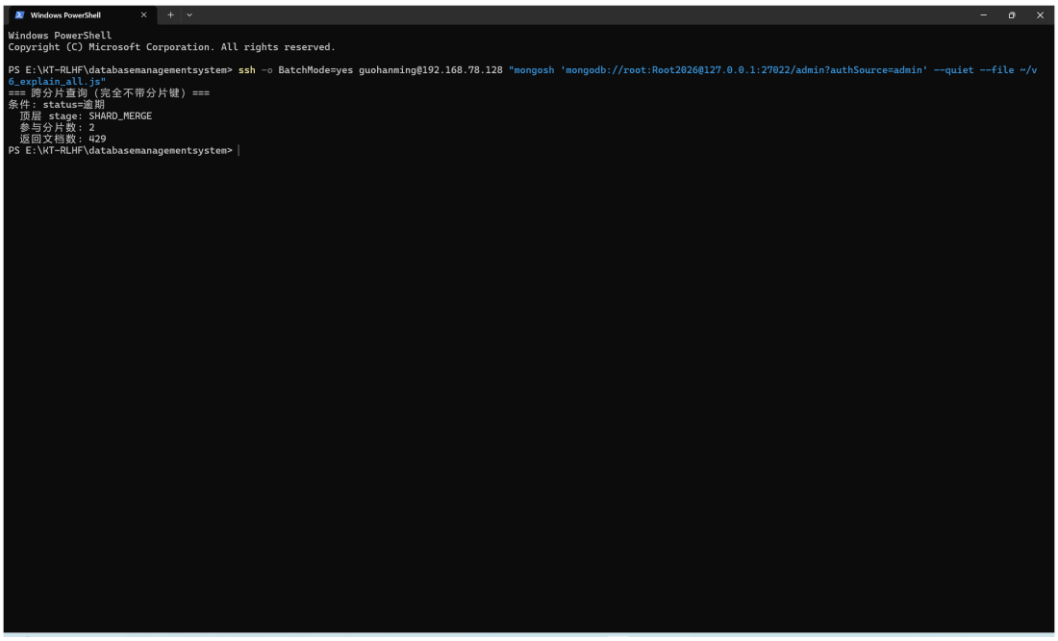


图 D-7　跨分片查询

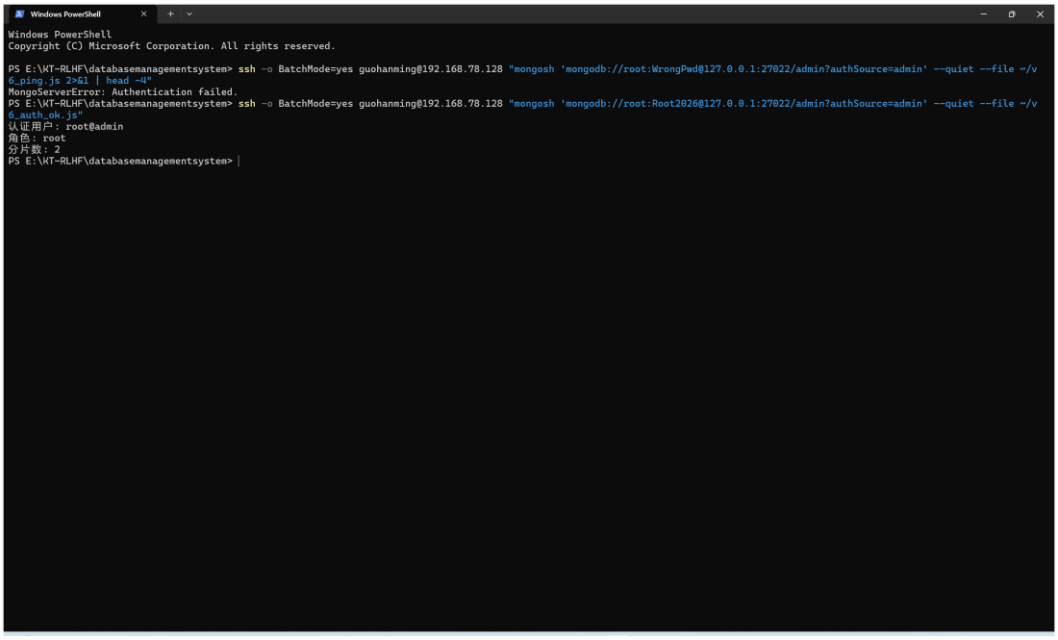


图 D-8　集群认证

附录 E　T7 验收

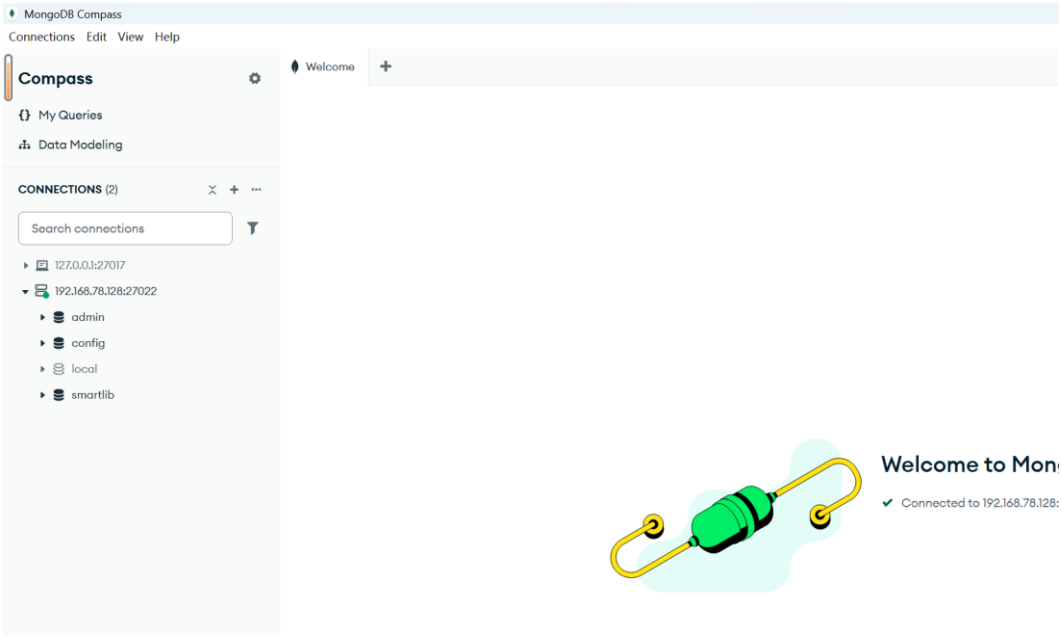


图 E-1　Compass 连接与库列表

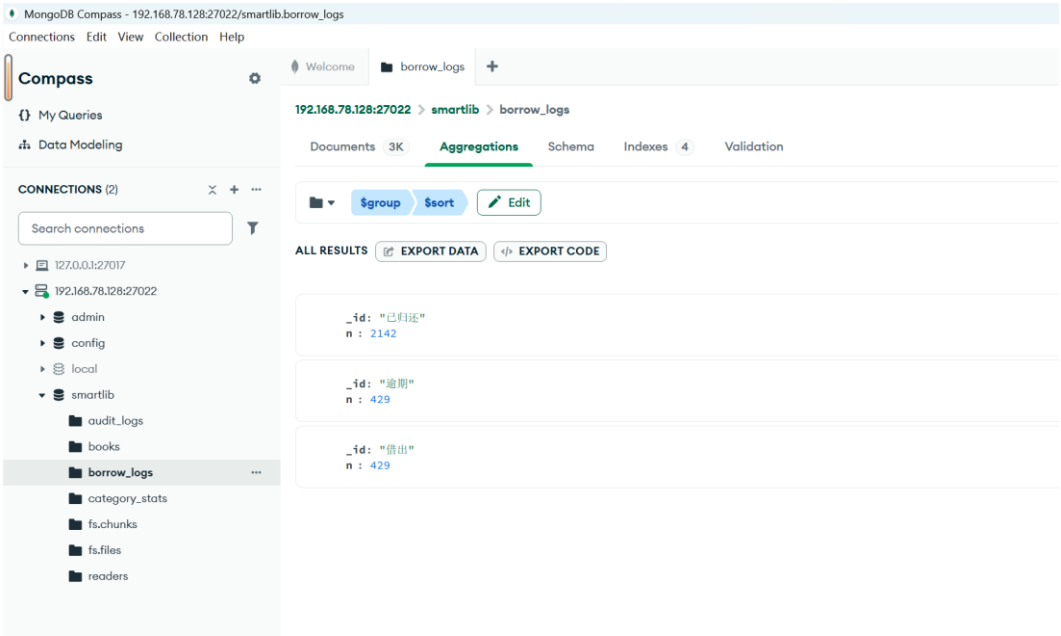


图 E-2　Compass 聚合对比

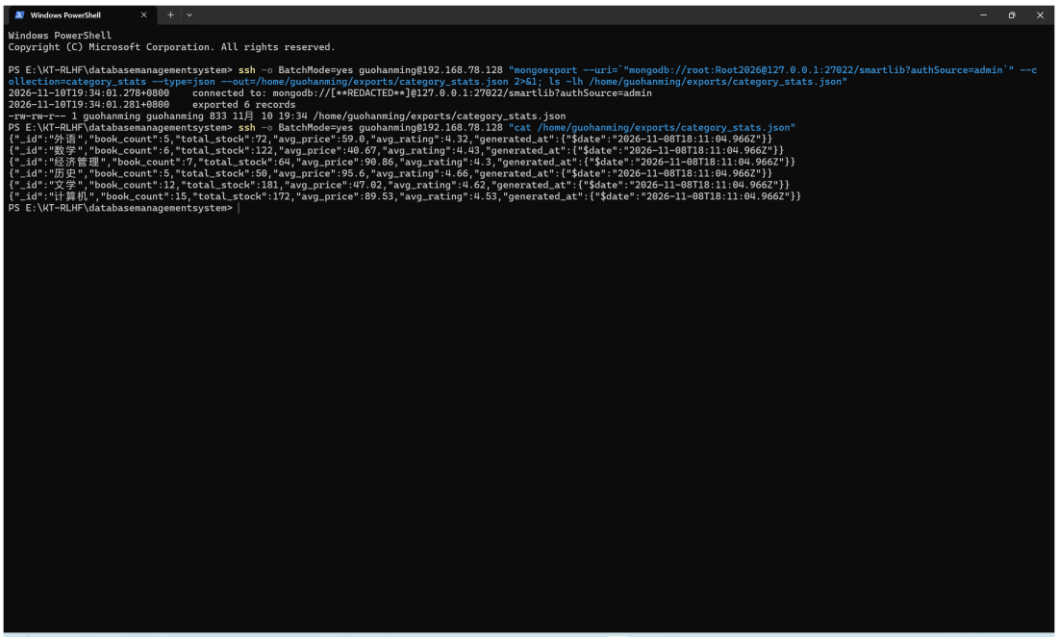


图 E-3　导出分类统计

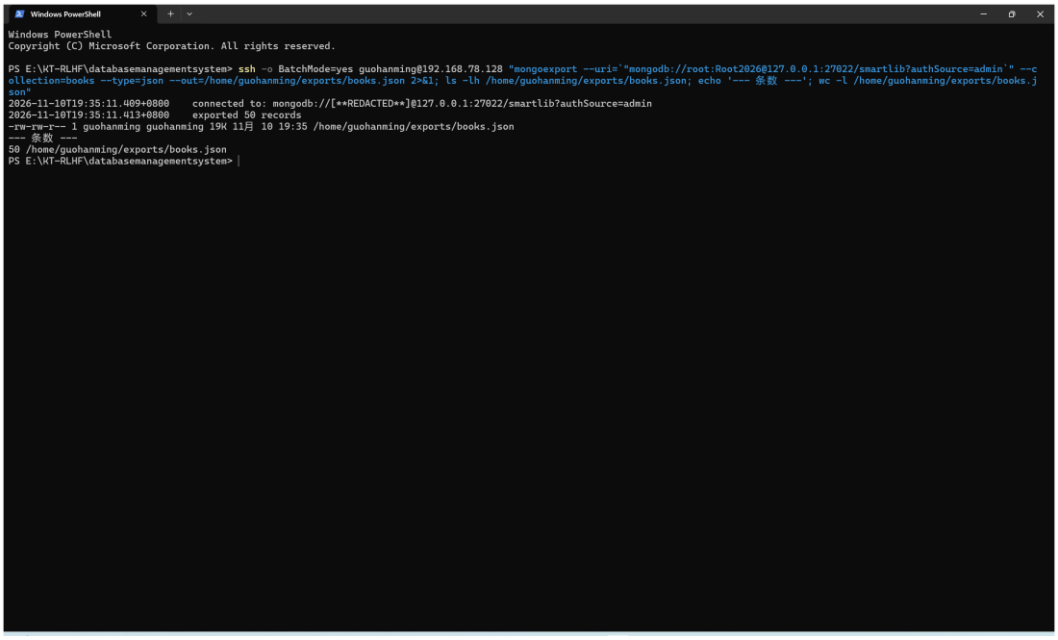


图 E-4　导出图书

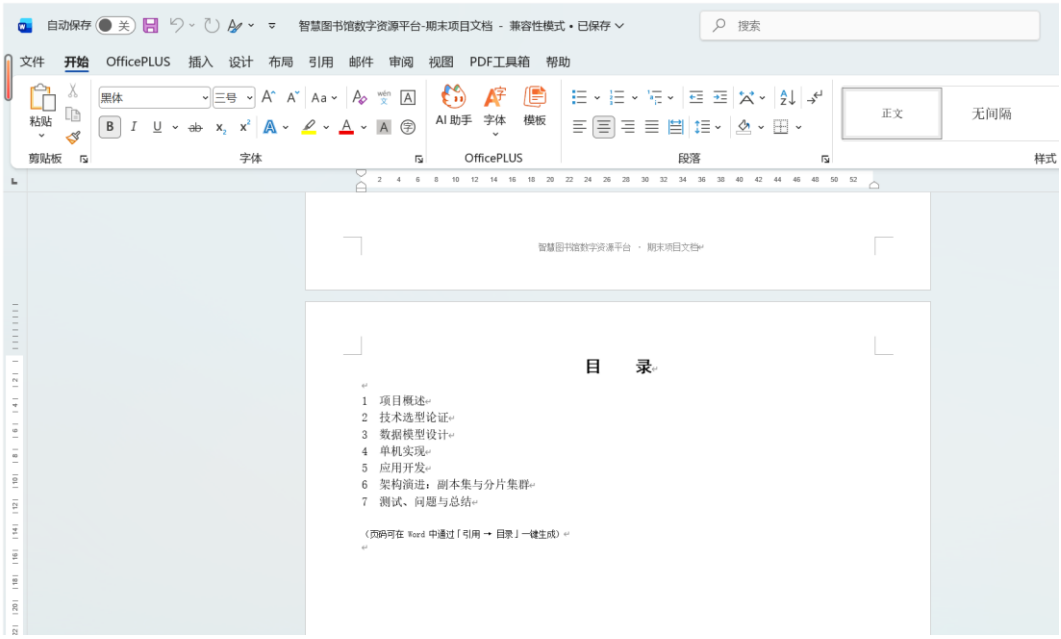

智慧图书馆数字资源平台-期末项目文档 - 兼容性模式 • 已保存
文件 开始 OfficePLUS 插入 设计 布局 引用 邮件 审阅 视图 PDF工具箱 帮助
AI 助手 字体 模板
正文 无间隔
剪贴板 字体 OfficePLUS 段落 样式
智慧图书馆数字资源平台 · 期末项目文档
目 录
1 项目概述
2 技术选型论证
3 数据模型设计
4 单机实现
5 应用开发
6 架构演进：副本集与分片集群
7 测试、问题与总结
（页码可在 Word 中通过「引用 → 目录」一键生成）

图 E-5　文档目录与页数